CLOUD COMPUTING PROJECT

Cloud-Based Industrial Equipment Monitoring and Fault Detection

Requirements specification

System design and implementation

Tools, diagrams and verification evidence

15 October 2026
AWS Asia Pacific (Sydney) / ap-southeast-2

Deployment: https://d1idggpc663p7f.cloudfront.net

Prepared from implemented source and recorded acceptance evidence. Source baseline: b31c746; documentation is committed separately.

Acceptance exception: initial-account password setup/recovery remains unresolved. Final actual AWS spend has not yet been reconciled. See sections 2.14 and 4.5.

Contents

Part 1. Requirements specification 4

1.1 Project identification and problem . . . 4
1.2 Objectives and boundaries . . . 4
1.3 Actors and user journeys . . . 4
1.4 Functional requirements and implementation traceability . . . 5
1.5 Telemetry contract . . . 6
1.6 Non-functional requirements . . . 7
1.7 Acceptance and submission completion . . . 7

Part 2. Design and implementation 8

2.1 Architecture and design decisions . . . 8
2.2 Physical deployment and networking . . . 8
2.3 Telemetry lifecycle and failure semantics . . . 9
2.4 Data model, indexes and retention . . . 9
2.5 Fault detection, notification and motor control . . . 10
2.6 Authentication, authorization and security . . . 11
2.7 Application API design . . . 12
2.8 Historical reporting and SQL . . . 12
2.9 React implementation . . . 13
2.10 Deployment, releases and operational workflow . . . 13
2.11 Recovery and later shutdown . . . 14
2.12 Verification results and interpretation . . . 14
2.13 Cost model and course coverage . . . 15
2.14 Known limitations and open acceptance items . . . 16

Part 3. Tools and technologies used 17

3.1 AWS service inventory . . . 17
3.2 Application languages and libraries . . . 18
3.3 Testing, validation and engineering tools . . . 19
3.4 Source map and reproducibility . . . 20
3.5 References . . . 20

Part 4. Verification evidence and screenshots 22

4.1 Evidence provenance . . . 22

4.2 Scenario coverage . . . 23
4.3 Selected visual evidence . . . 23
4.4 Demonstration sequence for an evaluator . . . 29
4.5 Submission readiness record . . . 30

Part 5. Architecture and design diagrams 31

Figure 1. System context . . . 31
Figure 2. Implemented AWS components . . . 32
Figure 3. Network and runtime boundary . . . 33
Figure 4. Telemetry commit and retry sequence . . . 34
Figure 5. Logical data model and access patterns . . . 35
Figure 6. Browser authentication and authorization . . . 36
Figure 7. Controls and fault state machines . . . 37
Figure 8. Historical reports and limited recovery . . . 38
Figure 9. Frontend software structure . . . 39
Figure 10. Build, deployment and evidence lifecycle . . . 40

Part 1. Requirements specification

1.1 Project identification and problem

Project: Cloud-Based Industrial Equipment Monitoring and Fault Detection. **Context:** an academic Cloud Computing project using AWS. **Deployment:** Asia Pacific (Sydney), `ap-southeast-2`. **Report date:** 15 October 2026.

Equipment monitoring needs a reliable way to collect measurements, identify sustained problems, present current conditions and inspect earlier operation. A direct connection from a device to a dashboard does not provide durable buffering, historical analysis or a clear recovery path. This project implements those responsibilities as separate cloud components and demonstrates them using three simulated motors.

One EC2 instance represents a factory gateway. It emits temperature, vibration, power and operating state for `motor-01`, `motor-02` and `motor-03`. The measurements are synthetic; no physical sensors or factory integration are claimed. All motors begin healthy. An operator deliberately introduces faults through the dashboard, observes the resulting readings and alerts, and then restores normal operation.

1.2 Objectives and boundaries

The system must demonstrate authenticated telemetry ingestion, queued processing, current and historical storage, deterministic fault detection, email notifications, a protected interactive dashboard and bounded historical SQL reports. Infrastructure must be reproducible through CloudFormation and the AWS CLI. Failure demonstrations must cover duplicate events, processing interruption, invalid messages and limited recovery.

The project uses the existing AWS Free Plan and a USD 50 service-usage ceiling before credits. The account initially had USD 100 credits recorded on 14 October 2026. Credits offset billing; they do not increase the agreed project ceiling. Notify at USD 20, 30 and 40, and obtain the user's agreement before stopping resources for spending reasons. Keep the deployment online until shutdown is requested. Budget alerts are notifications, not a hard spending cap.

Sydney replaces the original Mumbai deployment choice with the user's approval after managed-account restrictions prevented the required Mumbai resource creation. Kinesis Data Streams, Data Firehose and QuickSight are excluded. SQS provides buffering, S3 stores history, Athena queries it, and the React dashboard supplies visualization.

Outside scope: physical hardware, IoT Core/MQTT, machine learning, WebSockets, public self-registration, social federation, arbitrary SQL, editable fault thresholds, container orchestration, relational databases, multi-region failover, production industrial safety decisions and a general infrastructure administration UI.

1.3 Actors and user journeys

Actor	Required journey	Access boundary
Viewer	Sign in, inspect all motors and recent charts, filter alerts, request historical reports and download report CSV	Authenticated read/report routes; report results belong to the requesting principal
Operator	Perform viewer journeys, request individual faults/reset-all, invite users and change viewer/operator roles	Server-side operator enforcement; current group membership checked for privileged requests
Simulator gateway	Publish batches and poll/acknowledge requested motor modes	IAM instance role and signed ingestion requests; no human password
Infrastructure maintainer	Deploy/update, inspect logs and queues, demonstrate failures/recovery, export evidence and later perform authorized cleanup	Authorized AWS CLI identity; all operator AWS commands appended to the command file

The two initial team/evaluator accounts were provisioned with both viewer and operator roles. Creating a login account does not subscribe it to equipment email. SNS email subscriptions require their own confirmation. The confirmed project notification mailbox is `collegeproject@akshaykripalani.com`.

1.4 Functional requirements and implementation traceability

ID	Required behavior	Implementation and evidence
FR-01	Three initially healthy motors; readings every 10 seconds; gradual synthetic values; independent normal, overheat, vibration and disconnect modes	`simulator/main.py`; actual EC2 fault/recovery and bounded burst/stopped-motor tests
FR-02	Authenticate simulator with IAM; validate complete schema and registered IDs; return accepted only after queuing	`backend/ingestion.py`, `backend/domain.py`; signed malformed-input rejection and live pipeline tests
FR-03	Batch queued processing, retry safety, duplicates, older events, partial failures and DLQ isolation	`backend/processor.py`; transaction tests, duplicate/order live test, real poison-message retry/redrive
FR-04	Latest state and freshness, stopped versus offline, 24-hour recent history, 30-day archive and bounded object versions	Data table, API time filters and S3 lifecycle; stopped/offline, archive and retention-configuration evidence
FR-05	Activate after three consecutive threshold breaches; resolve after three healthy readings; one notification intent per episode; retain alert history	`backend/domain.py`, processor and notifier; actual fault activation/resolution, confirmed SNS acceptance and human inbox receipt
FR-06	React overview, device charts, alerts, reports, controls, users; loading, errors, empty results and stale data; 10-second refresh	`frontend/src/`; 23 controlled browser scenarios, four deployed journeys and final hosted-login/chart smoke
FR-07	Cognito code/PKCE login, access-token API protection, sign-out/expiry, viewer/operator backend enforcement	`frontend/src/auth.ts`, JWT authorizer and backend role checks; hosted PKCE, direct API rejection and role-revocation tests; initial-account recovery remains open
FR-08	Three predefined date/device reports over S3 using Glue/Athena; bounded inputs, scan limits, asynchronous status/results	`backend/domain.py:report_sql`, `backend/api.py`; known-data cross-checks for all three reports and real UI charts/CSV
FR-09	Useful bounded logs, Lambda and queue alarms, operational notification and deliberate failure evidence	CloudWatch log groups, ten alarms, SNS; recorded health snapshot and intentional processing-failure logs
FR-10	CloudFormation/CLI deployment, private frontend hosting, restricted EC2 networking, version restore and separate-table reconstruction	`infra/`, deployment/recovery scripts; template validation, cache/private-hosting tests and measured limited recovery
FR-11	Shared independent motor controls with request/run matching, requested versus applied modes, reset behavior and timeout visibility	Controls table, API, independent simulator polling and dashboard; actual acknowledgements, reset, disconnect and unaffected-motor checks

The table maps feature implementation to evidence. A passing test verifies its defined scenario, not every possible future fault. Initial-account access is an explicit acceptance exception rather than an omitted requirement.

1.5 Telemetry contract

Field	Meaning and validation
`schemaVersion`	Integer 1; no implicit alternative schemas
`eventId`	Stable across retries; 1-160 characters from the permitted identifier alphabet; run/device/sample identity
`deviceId`	One of the three registered motor IDs
`observedAt`	UTC ISO-8601 event time, normalized to milliseconds; accepted within last 24 hours with up to 60 seconds future skew
`temperatureC`	Finite number from -40 to 200 degrees Celsius
`vibrationMmS`	Finite number from 0 to 100 millimetres/second
`powerWatts`	Finite number from 0 to 100000 watts
`operatingState`	`running` or `stopped`

The request body contains only `readings`, an array of 1-30 readings. Each reading contains exactly the documented fields. Booleans are rejected as numeric values, as are NaN/infinity, duplicate IDs within a batch and unknown devices. Receipt time is added by ingestion; an anomaly label is not accepted as a measurement field. API request bodies are bounded to 65536 bytes.

1.6 Non-functional requirements

Area	Target or constraint	Implemented measure / qualification
Latency	Normal-load ingestion-to-current-state target under 30 seconds	Recorded sample 5.063 seconds including polling; not a percentile SLA
Freshness	Poll relevant views every 10 seconds; offline after 60 seconds without a valid processed receipt	Server timestamps, browser clock progression, observed/last-seen distinction and stale warning
Integrity	Retries must not recount events, regress latest state or repeatedly activate the same episode	Stable IDs, canonical fingerprints, conditional archive writes and DynamoDB transactions
Security	Authenticated ingestion and human APIs, private history/assets at origin, role-based mutations, encryption and no stored credentials	IAM/SigV4, Cognito/JWT, CloudFront OAC, HTTPS, scoped roles and command-log filtering
Cost	Small bounded demonstration within agreed service-usage budget	Small ARM instance, no NAT, on-demand data services, retention, query limits and budget emails; no automatic spending stop
Operability	Reproducible infrastructure, bounded logs, failure isolation, command audit and future cleanup procedure	Three CloudFormation stacks, seven-day logs, DLQ, logged CLI wrapper and shutdown runbook
Usability	Desktop/mobile, clear states, accessible controls and minimal entry copy	Responsive layout, keyboard/reduced-motion support and automated axe checks; full manual accessibility audit not claimed
Recovery	Demonstrate object-version restoration and reconstruction without disturbing live state	Disposable recovery table, original times, deduplication and no notification publication; no regional DR guarantee

1.7 Acceptance and submission completion

Acceptance uses normal operation, individual faults, reset/disconnect, stale versus stopped readings, role checks, chart pagination, historical report accuracy, buffering, DLQ recovery, private hosting/cache updates and limited reconstruction. The original PRD acceptance scenarios remain authoritative. Recorded evidence is indexed in Part 4 rather than replaced by this document.

Implementation acceptance is supported by 64 backend/infrastructure tests, 14 distinct live AWS scenarios consolidated across runs, 23 controlled browser scenarios, four deployed browser journeys and a later desktop/mobile smoke. These counts overlap in feature coverage and must not be added into a single number of independent end-to-end tests.

Before a fully unqualified final demonstration, resolve the initial account's password setup/recovery problem and verify that person's sign-in. Before reporting a final actual project cost, reconcile the delayed AWS billing data with the project period and shared-account activity. Full production teardown is deliberately deferred until the user requests shutdown.

Part 2. Design and implementation

2.1 Architecture and design decisions

The platform separates equipment simulation, ingestion, durable buffering, processing, live queries, historical analysis and browser delivery. This lets a processing interruption leave accepted telemetry queued while the dashboard and ingestion endpoint remain available. DynamoDB serves predictable current/recent access patterns; S3 and Athena serve historical aggregation without making every live refresh run SQL.

The implemented system has four Python Lambda entry points: Ingestion, Processor, ApiBackend and Notifier. Controls and report coordination are routes within ApiBackend, not separate deployed control/report functions. A scheduled Notifier publishes durable notification intents; Processor does not directly send email. These details refine the PRD's conceptual diagram and are shown in Figure 2.

Decision	Rationale	Trade-off
Single EC2 gateway, three simulated motors	Demonstrates compute, networking and shared remote controls without hardware	Gateway failure affects all motors; no EC2 fleet failover
API Gateway and SQS FIFO	IAM entry point, backpressure and ordered processing	One `factory` message group serializes this demonstration; not a high-throughput multi-factory design
DynamoDB plus S3	Low-latency live reads plus inexpensive analytical archive	Cross-service writes need replay-safe design; no single global transaction
Threshold rules with consecutive counters	Repeatable fault demonstrations and understandable decisions	No learned anomaly model or predictive maintenance
Separate notification outbox	Alert recording does not depend on immediate SNS availability	Delivery is asynchronous and may duplicate across a publication/recording crash
Athena predefined queries	Demonstrates data engineering with bounded scan cost	Seven-date queries, 30-day horizon and application quota restrict exploration
Static React through CloudFront/private S3	HTTPS, caching and independent UI releases	Browser rendering and polling; no SSR or push telemetry
Three CloudFormation stacks	Separate artifact bootstrap, application platform and simulator lifecycle	Deployment order and cross-stack outputs must be managed

Figure 1 presents system context; Figure 2 shows AWS components; Figure 3 shows the simulator network boundary. Full-size diagrams and editable sources are in the diagram appendix.

2.2 Physical deployment and networking

The live application entry point is https://d1idggpc663p7f.cloudfront.net. Protected application requests go to https://9ojy2mmg65.execute-api.ap-southeast-2.amazonaws.com. The regional resources are in Sydney; CloudFront supplies edge delivery. Authentication redirects to the project's Cognito domain and returns to the exact configured `/callback` URL.

`ccproj-artifacts` owns the private versioned release bucket. `ccproj` owns the platform services, storage, identity, scheduling and monitoring. `ccproj-simulator` owns the gateway instance and VPC network resources. Resource identifiers and stack outputs are recorded in `docs/ccproj-deployment.json`; the report uses logical names where physical identifiers add no design information.

The simulator VPC uses `10.42.0.0/24`, with a public subnet `10.42.0.0/26` and an Internet Gateway/default route. The instance uses a public IPv4 address to reach AWS service endpoints. There is no NAT Gateway and no inbound security-group rule. Outbound HTTPS and DNS support API, DynamoDB, SSM, logs and software/release access. A public subnet does not imply that SSH or an application listener is exposed.

The instance is ARM `t4g.micro` with an encrypted 8 GiB gp3 root volume, standard CPU-credit mode and required IMDSv2. A systemd-supervised Python simulator starts at boot. SSM supplies management and release updates without opening SSH. Lambda uses Python 3.12 ARM64, 256 MiB memory and 30-second timeouts in the generated templates. Application Lambda functions are not deployed into this custom simulator VPC.

2.3 Telemetry lifecycle and failure semantics

1. The simulator generates a smooth synthetic reading for each reporting motor. The event ID remains stable if transmission must retry. A disconnected motor stops reporting while the other motors continue.
2. The instance role signs `POST /telemetry` with SigV4. API Gateway requires IAM authorization for this route. Human Cognito tokens do not substitute for that permission.
3. Ingestion parses and validates the full batch. It normalizes times, adds receipt time and computes a canonical SHA-256 batch identity. Only after SQS accepts the message does it return HTTP 202. This response means queued, not processed.
4. The FIFO queue uses the `factory` message group and explicit batch deduplication ID. Its retention is four days, visibility timeout 180 seconds, and redrive limit three receives. The matching FIFO DLQ retains messages for 14 days. Queue encryption uses SQS-managed encryption.
5. Processor validates the envelope relative to its original receipt time, allowing a legitimately accepted message to wait in a backlog. It checks event fingerprints before archival, then writes compressed date-partitioned history and processes each reading into DynamoDB.
6. S3 archive keys have the shape `telemetry/day=YYYY-MM-DD/{batchId}.json.gz`. Newline-delimited JSON includes receipt time. A cross-midnight batch is split by observed date. Deterministic gzip content and a conditional `IfNoneMatch` write prevent retries from creating unnecessary versions of the same batch object.
7. Each reading commits a DynamoDB transaction containing its deduplication marker, recent-reading item, revised device state and any alert transition/notification intent. Conditional revision checks retry up to five times when state changes concurrently. The marker stores a canonical fingerprint: reusing an ID with different measurements is an error.
8. Only a strictly newer observed timestamp replaces the current measurement or advances fault counters. Receipt time tracks liveness separately. An older event can be archived and stored as recent history without regressing the device state. A duplicate does not create another reading or episode.
9. Processor acknowledges a queue message after required archive and transactional operations complete. On the first failure in a FIFO batch, it returns that message and all subsequent unprocessed message IDs in `batchItemFailures`. Earlier committed work is safe to encounter again because event markers suppress recounting.

The event source mapping batch size is ten. A single batch can fail after an archive write or after some per-reading transactions have committed. Retries reconcile those partial outcomes; there is no atomic S3-plus-DynamoDB commit. SQS deduplication alone is insufficient because it has a bounded window; DynamoDB markers extend application-level retry handling to 31 days. The system does not claim exactly-once processing forever or permanent deduplication after markers expire.

2.4 Data model, indexes and retention

The on-demand Data table uses string `pk` and `sk` keys. Different item types support explicit query patterns without a relational join. Device metadata includes registered identity and configured thresholds in application configuration; latest state and rule counters are persisted. Controls use a separate on-demand table so requested scenarios are not mistaken for measured equipment conditions. Figure 5 maps the records and access patterns.

Store/item	Key or layout	Principal contents and use	Retention
Data: device state	`pk=motor-01`, `sk=STATE`	Latest reading, lastSeen, revision, per-rule active/counter state	Current state; no expiring history window
Data: recent reading	`pk=deviceId`, `sk=READING#{observedAt}#{eventId}`	Measurement and receivedAt; time-range query and pagination	TTL at observed time +24 hours
Data: event marker	`pk=EVENT#{eventId}`, `sk=META`	Canonical fingerprint and archive key	TTL 31 days from processing
Data: alert episode	`pk=deviceId`, `sk=ALERT#{alertId}`	Rule, activation reading/time, active/resolved and resolution time	TTL 30 days
Data: notification intent	`pk=NOTIFY`, `sk=alertId`	Device/rule/reading, delivery state, SNS message ID	TTL 30 days
Data: report metadata	`pk=REPORT#{queryId}`, `sk=META`	Owner, validated parameters and query association	TTL seven days
Data: report quota	`pk=QUOTA#{UTC-date}`, `sk=REPORTS`	Atomic daily report-start counter	TTL two days
Controls: motor request	`pk=deviceId`, `sk=CONTROL`	Requested/applied modes, request/run IDs, acknowledgement timestamps	Current control record
Controls: gateway run	`pk=GATEWAY`, `sk=RUN`	Simulator run identity and start time	Current run record
S3 archive	Date partition / batch gzip object	Historical measurements including receipt time	Current and noncurrent versions bounded to 30 days
S3 report results	Private Athena output objects	Query result files and metadata	Seven-day lifecycle

The sparse Data `Work` GSI partitions on `workStatus` and sorts on `workAt`. Only pending notification intents carry those attributes. After a successful publish is recorded, the attributes are removed, taking the item out of pending work queries. This avoids repeatedly scanning every device/history item.

DynamoDB TTL and S3 lifecycle expiration are asynchronous. API bounds enforce the recent window independently of physical removal. The release-artifact bucket has a 90-day lifecycle and frontend assets a 365-day lifecycle; these differ from telemetry retention. Stateful resources use retention policies to preserve established data across stack deletion/replacement. Operators must deliberately remove retained resources during a later authorized shutdown.

2.5 Fault detection, notification and motor control

Each device has independent temperature and vibration rule state. Temperature is breached only above 75 degrees Celsius; vibration only above 4.5 mm/s. Equality is healthy. A breached reading increments the above counter and resets the healthy counter. A healthy reading does the reverse. Counters saturate at three. The third consecutive breach activates an episode; the third consecutive healthy reading resolves it. An intervening opposite result resets the relevant sequence.

Activation writes an alert and notification intent in the same DynamoDB transaction as the triggering reading/state. Sustained breach does not create repeated activation intents. Resolution updates the existing alert instead of deleting it, and no recovery email is sent. Temperature and vibration may be tracked independently even though the operator selects one simulator mode at a time.

EventBridge Scheduler invokes Notifier once per minute. The worker reads up to 50 pending intents per invocation and checks confirmed SNS subscriptions. With none confirmed, it records `pending_confirmation` while retaining the work item. A successful SNS publish records `sns_accepted`, message ID and publication time, then removes the pending-index attributes. Failed attempts remain for a later run and raise an operational error. SNS uses a project KMS key for topic encryption.

`sns_accepted` describes provider acceptance, not inbox delivery. The actual demonstration has separate human confirmation that both fault emails arrived. A crash between SNS accepting a publication and DynamoDB recording success can cause another email on retry. Notification timing includes the scheduler delay and external email delivery, so it is not the same as telemetry processing latency.

Operator control routes write requests instead of directly changing device measurements. A request identifies the motor, requested mode, request ID and current gateway run. A separate simulator thread polls controls about every five seconds, applies a matching request and conditionally acknowledges it. The dashboard shows requested and applied states separately. A missing acknowledgement after 60 seconds displays a timeout; the request remains available for reconciliation.

Overheat and vibration modes gradually alter the generated measurements. Normal mode brings them toward baseline over about 30 seconds, after which the three healthy readings resolve an active rule. Disconnect suppresses only the selected motor's transmission. Reset-all issues three independent normal-mode requests and does not erase alert history. A stopped motor still reports with zero power and can be online; a disconnected motor eventually becomes offline. Figure 7 shows these state transitions.

2.6 Authentication, authorization and security

People and the gateway have separate trust paths. The gateway uses its EC2 IAM role and signed AWS requests. People sign in with the Cognito user pool. The browser is a public OAuth client without a secret and uses authorization code with PKCE S256, random state and exact registered redirect URLs. `openid email profile` scopes support sign-in and the configured protected routes. Figure 6 shows the interaction.

`oidc-client-ts` manages the browser exchange and session. Tokens live in per-tab session storage, not source/config files. Access and ID tokens last ten minutes; refresh tokens last one day. The API receives the access token, not the ID token. API Gateway validates the JWT issuer, audience/client and supported scope. ApiBackend enforces viewer/operator membership. Privileged operations additionally check current Cognito groups, preventing a still-valid older operator token from keeping removed privileges. Operators cannot remove their own operator role.

Read requests can renew authentication once after a 401. Mutations are not automatically replayed when their response is uncertain: retrying a control or invitation could repeat an action whose first outcome is unknown. Sign-out clears local state, revokes the refresh session where supported by the implemented flow, and visits hosted logout. Callback handling removes authorization-code parameters from the visible URL.

The API allows only predefined report identifiers and validated values. Each report metadata item records its requesting principal; another authenticated user cannot retrieve it merely by guessing its ID. Recent-history cursors are validated for the requested motor and key shape. Unknown device IDs and inappropriate roles are rejected server-side.

S3 buckets block public access and enforce secure transport. Frontend objects are read through CloudFront Origin Access Control; raw S3 origin access is denied. Historical and Athena-result buckets remain private. At-rest encryption covers S3, DynamoDB, EBS, queues and the SNS topic. IAM roles limit runtime service actions/resources; no access keys are embedded in the simulator or browser.

CloudFront sets CSP, HSTS, frame denial and content-type protection. The policy allows scripts and stylesheet elements from the same origin. Recharts requires inline style attributes for responsive charts, so `style-src-attr 'unsafe-inline'` is permitted specifically for that rendering need. This choice does not permit arbitrary inline scripts. Automated security/configuration checks are useful evidence, not a penetration-test certification.

The append-only CLI wrapper records timestamp, purpose, non-secret arguments and exit status, but not AWS responses. Sensitive inline inputs are rejected; protected temporary input files are used where necessary. Documentation

and screenshots exclude passwords, tokens and mailbox messages.

2.7 Application API design

All human routes below require Cognito access tokens. Viewer or operator membership permits read/report access; controls and users require operator. Telemetry alone uses IAM. JSON responses set `Cache-Control: no-store`.

Method and route	Request / result	Constraint
`POST /telemetry`	`{readings:[...]}`; 202 accepted	IAM signed; validated 1-30 item batch
`GET /devices`	Three device views and server time	State, freshness, thresholds and requested/applied control
`GET /devices/{id}`	One device view	Registered ID only
`GET /devices/{id}/readings`	`from`, `to`, `limit`, `cursor`; readings and nextCursor	Last 24 hours; limit 1-1000 per page
`GET /alerts`	Optional device/time/state filters; alerts and subscription counts	Last 30 days; bounded history response
`POST /controls/{id}`	`{mode}`; 202 request record	normal/overheat/vibration/disconnect
`POST /controls/reset-all`	Per-device requests	Operator; acknowledgement remains asynchronous
`POST /reports`	`{report, start, end, deviceId?}`; report ID/status	Predefined type, validated dates and daily quota
`GET /reports/{id}`	Status, parameters, scanned bytes, result rows/error	Requesting owner only
`GET /users`	Cognito accounts and viewer/operator groups	Operator; current bounded list up to 100 users
`POST /users`	`{email, roles}`; Cognito invitation	Operator; generated temporary password sent by Cognito
`PUT /users`	`{email, roles}`; role assignment	Operator; cannot remove own operator access

The API contract in `docs/API.md` is the integration reference. Validation, permission and query failures are shown explicitly by the UI; a successful request is not inferred from a button click or optimistic mode label.

2.8 Historical reporting and SQL

Glue defines the JSON archive schema with projected date partitions, avoiding a continuously running crawler. Athena scans the compressed history using a dedicated workgroup. The backend generates SQL from a fixed report map. Date and device parameters are validated before execution; the browser never supplies raw SQL.

Report	Grouping	Calculated values
Temperature	Motor and observed date	Average temperature, maximum temperature, reading count
Vibration	Motor and observed date	Count of readings above 4.5 mm/s

Report	Grouping	Calculated values
Power	Motor and observed date	Average power for running readings and count of running readings

A common query stage deduplicates `eventId` with `row_number()` ordered by receipt time. This handles a resent measurement appearing in more than one archive envelope. Vibration counts breached readings, not the number of activated fault episodes. Stopped readings are excluded from the running-power average.

A report spans one to seven inclusive dates within the last 30 days. A 10 MiB workgroup scan cutoff bounds each query; an application-wide atomic quota allows 100 report starts per UTC day. These controls reduce exposure but do not form a complete monetary cap. Query state is asynchronous: React polls about every three seconds, then displays charts, tables and CSV when complete, or a clear failure/empty state. Report results are stored privately in S3 and exposed through the authorized backend. Figure 8 combines the reporting and recovery paths.

2.9 React implementation

`main.tsx` mounts the React application. `App.tsx` manages authentication, navigation and hash routes for overview, device detail, alerts, reports and users. `pages.tsx` provides the views; shared controls and chart/display elements live in `components.tsx`. `api.ts` centralizes protected requests, `auth.ts` handles OIDC, `hooks.ts` supplies polling/clock behavior, and typed contracts are in `types.ts`. CSS is separated into tokens, application styles and entry-page styles.

Overview shows motor counts, measurements with units, active faults and freshness. Device detail plots temperature, vibration and power through Recharts. Historical points are paginated in pages of up to 1000, then updated with a 30-second overlap and event-ID deduplication. Display history is bounded to 24 hours and 10000 points. Gaps longer than 25 seconds are represented as gaps rather than a falsely continuous line. Threshold references help interpret the temperature/vibration charts.

Live polling runs every ten seconds on relevant visible pages and pauses when the document is hidden. Requests are cancelled when their view/range becomes obsolete. A failed refresh preserves the last successful readings while showing an error/stale indication. A progressing browser clock uses server-time offset so an old response becomes visibly stale even when no new response arrives. Network requests have bounded timeouts.

Alerts support device, state and 1/7/30-day filters, retained resolution history and honest notification states. Reports validate the date window, show asynchronous progress and generate CSV downloads. Operators see fault/reset controls and user management; viewers do not. Backend enforcement remains decisive. Requested/applied controls and timeouts remain visible during remote operation.

The entry screen was revised to a concise sign-in card with collapsed initial-account help, following the user's preference against promotional copy. Responsive navigation supports mobile screens, controls have accessible names and landmarks, and reduced-motion preferences are handled. Automated axe checks cover tested entry, monitoring, reports, alerts and account views, but do not replace manual assistive-technology evaluation.

2.10 Deployment, releases and operational workflow

The generator produces three ordinary CloudFormation JSON templates. Resource metadata records purpose and constraints. Validation combines cfn-lint, CloudFormation Guard, code linting and tests. Deployment checks the AWS account and region rather than relying on an assumed default profile.

The workflow is: validate source/templates; package versioned backend and simulator releases; create/update artifact storage; deploy the platform with artifact keys; provision the simulator with platform outputs; seed the initial Cognito groups/accounts; generate public non-secret frontend configuration; build React; publish static assets; invalidate CloudFront and verify the served release. The EC2 updater waits for cloud-init before changing the running service, preventing a boot/update race.

Frontend publishing is separate from backend infrastructure deployment. Vite builds content-hashed bundles; `publish_frontend.py` uploads them and updates the entry point. Old hashed assets are preserved during a release so an existing page can finish loading its referenced files. `index.html` is not treated as an indefinitely cached immutable asset. `/config.json` supplies public endpoint, region and Cognito identifiers and contains no credentials. The recorded release was checked by fetching every served build asset and comparing SHA-256 hashes.

All operator AWS CLI commands pass through `scripts/aws_cli.py` and append to `AWS_COMMANDS.txt`, including unsuccessful operations. This is an execution audit, not a template of unexecuted commands. Runtime Lambda/EC2 SDK calls are not CLI commands and are not represented as individual entries in that file. Deployment and test scripts retain their own evidence outputs without writing secrets into the log.

CloudWatch captures seven-day Lambda/simulator logs and ten operational alarms for function/queue health. Equipment alarms are application rules; CloudWatch alarms monitor the platform. During failure demonstration, repeated poison-message processing errors were observed, the message entered the DLQ, its invalid envelope was inspected/corrected, and corrected work was redriven. Consumer interruption demonstrated continued queue acceptance and subsequent catch-up.

2.11 Recovery and later shutdown

Recovery is deliberately limited and independently verifiable. The object-version test overwrites/deletes a known test object and restores its prior version. `scripts/recovery.py` reads a bounded archive prefix, decompresses NDJSON, deduplicates event IDs and sorts by original observed time/event ID. It reconstructs state with original receipt/observation times in a separate disposable CloudFormation-managed DynamoDB table. It never writes the live state table and never calls SNS. Its archive listing is bounded to 10000 objects. This establishes a small reconstruction demonstration, not automated regional disaster recovery.

The recorded known three-device reconstruction and object restore took 70.641 seconds before verification/cleanup; the entire test took 137.105 seconds. Those timings depend on the small chosen fixture and AWS control-plane response and are not a promised recovery-time objective for 30 days of data.

No full live-stack teardown was run because the project must remain online. `docs/SHUTDOWN.md` specifies evidence export, ownership snapshots, simulator/platform/artifact stack deletion, and explicit handling of retained buckets, tables, user pool and KMS key. Versioned bucket cleanup must remove object versions and delete markers. KMS deletion has a required waiting period. Only project-owned resources may be removed, and billing must be rechecked after propagation delay.

2.12 Verification results and interpretation

Verification layer	Recorded outcome	What it establishes
Backend/infrastructure tests	64 passed, plus Ruff, cfn-lint and Guard on all three templates	Domain, handler integration, controls, audit and infrastructure constraints in tested cases
Real AWS suite	14 distinct latest-per-scenario passes across several runs	Actual services, queue buffering/DLQ, IAM/Cognito, reports, private hosting/cache and limited recovery
Controlled browser suite	23 passed after entry-page refinement	Repeatable UI success/error/empty/auth/accessibility states using deterministic fixtures
Deployed browser full run	Four journeys passed; 275.73 seconds including setup/cleanup	Real hosted PKCE, monitoring/controls/recovery, reports/CSV, role persistence and logout
Final published smoke	Desktop/mobile hosted-login and live-chart smoke passed	Final entry-page release integration after the earlier full run

The latest-per-scenario AWS summary is not a claim that all cases passed in one uninterrupted run. Original failed/interrupted runs remain in `docs/`. A browser selector initially chose a hidden Cognito form; it was corrected to use

visible fields and the deployed journey rerun. Backend/deployment fixes included per-queue policies and waiting for cloud-init. The consolidated evidence preserves failures rather than silently discarding them.

The final AWS-stage snapshot recorded three healthy motors, enabled consumer, empty queue/DLQ counts, an online SSM-managed instance and ten OK alarms. This is a point-in-time snapshot. The demonstrated fault notifications have SNS message IDs plus the user's confirmation that both arrived. Automated tests cannot guarantee future AWS availability, external mailbox delivery, every concurrency pattern or every future dependency/policy change.

2.13 Cost model and course coverage

The captured 14 October 2026 Sydney model estimates USD 26.93 for 730 hours and USD 12.39 for 14 days, before credits/free allowances. It assumes three motors every ten seconds, 788400 monthly readings, bounded reports and a USD 5 allowance for variable services. The component model is in `evidence/budget-model.json`; pricing was queried through the logged AWS CLI during implementation.

Monthly component	Estimated USD
EC2 t4g.micro	7.7380
One public IPv4 address	3.6500
8 GiB gp3 volume	0.7680
One KMS key and ten standard alarms	2.0000
DynamoDB transactional writes	3.3586
DynamoDB reads and control polling	0.5617
S3 archive PUT operations	1.4454
FIFO SQS operations	0.3942
Lambda requests and duration allowance	2.0119
API/logs/storage/reports/frontend allowance	5.0000
Rounded model total	26.93

These are historical planning estimates, not a newly queried price quote or final invoice. Actual usage must be measured. A recorded USD 0 budget reading was affected by billing delay and does not demonstrate zero cost. Budget thresholds are account-wide for the project period, so unrelated teammate spending may trigger them. Continued operation can exceed USD 50 over time; notification does not stop that automatically.

The implementation maps to the PRD's course outline through architecture, IAM, S3, EC2, DynamoDB, VPC, automation, CloudFront caching, SQS decoupling, Lambda/API serverless work, Glue/Athena data engineering and limited recovery. These are twelve direct practical areas out of seventeen, approximately 71%, plus partial security and monitoring/availability coverage. Orientation is supporting context; inter-network connectivity and certification preparation are not implemented features. This is a mapping to the supplied outline, not an independently verified college grading decision.

2.14 Known limitations and open acceptance items

The platform is an academic demonstration with a single simulated gateway. It has no hardware validation, safety certification, regional failover, full disaster-recovery automation, exactly-once email guarantee or unbounded historical analysis. DynamoDB TTL and S3 expiration are asynchronous. Single-group FIFO and small Lambda/instance settings suit the specified workload, not an established production capacity claim.

Initial account sign-in remains unresolved. The user's latest report was that a chosen password was rejected and reset mail did not arrive. The last read-only diagnostic still showed `collegeproject@akshaykripalani.com` enabled and `FORCE_CHANGE_PASSWORD`, with verified email and last modification at 15 October 2026, 00:27:37 IST. This suggests first-password completion has not been established for that account; the exact cause still requires diagnosis. Real automated hosted-login tests used disposable accounts and do not prove that this initial user's permanent password works.

Cognito's configured default email sender is `no-reply@verificationemail.com`. A fresh invitation had been accepted by AWS, forwarded by the mailbox routing service and found in destination Spam, then moved to Inbox. That delivery investigation does not prove later password-reset delivery or close the reported password problem. No password or reset code is included in submission evidence. The other initial account's first-access completion is also not claimed by this report.

Final actual-cost reconciliation, complete manual accessibility evaluation, production-scale capacity testing and a full user-authorized live teardown have not been performed. The report documents implemented behavior, bounded measured demonstrations and these remaining limitations separately.

Part 3. Tools and technologies used

3.1 AWS service inventory

Service/tool	Use in this implementation	Why it is needed
Amazon EC2 and EBS	One ARM gateway with an encrypted gp3 boot disk	Runs the persistent motor simulator and supervised process
Amazon VPC, subnet, route table, Internet Gateway and security group	Restricted public-subnet egress, no inbound rules	Network boundary and connectivity without NAT Gateway cost
AWS Systems Manager	Managed-node health and remote release commands	Operates the gateway without SSH ingress
Amazon API Gateway HTTP API	IAM telemetry entry and JWT-protected human routes	Separates device and browser authentication at the API boundary
AWS Lambda	Ingestion, Processor, ApiBackend and Notifier	Event-driven backend compute without continuously running application servers
Amazon SQS FIFO and FIFO DLQ	Ordered queue, retry/partial failure and failure isolation	Decouples producers from storage/processing and supports recovery demonstrations
Amazon DynamoDB	Data/Controls tables, transactions, TTL and sparse Work index	Current state, recent history, controls, deduplication, report ownership and notification intent
Amazon S3	Release artifacts, private frontend, compressed history and private query outputs	Different policies/lifecycles for application assets, historical records and results
Amazon Athena	Three bounded on-demand SQL reports	Historical aggregation without a continuously running database engine
AWS Glue Data Catalog	Archive table/schema and date projection	Makes S3 JSON history queryable by Athena without a persistent crawler
Amazon SNS	Confirmed equipment and operational email subscription	Publishes asynchronous fault and platform notifications
Amazon EventBridge Scheduler	One-minute notification worker invocation	Retries durable pending notification work independently of ingestion
Amazon Cognito	Hosted login, user pool, public PKCE client, viewer/operator groups and invitations	Human account and session management
Amazon CloudFront and Origin Access Control	HTTPS static delivery, caching, origin restriction and headers	Distributes React while keeping S3 origin private
AWS IAM and STS	Service/instance roles, authorization and active account identity check	Grants runtime access without embedded access keys
AWS KMS	SNS topic encryption key	Protects stored notification messages using managed key access

Service/tool	Use in this implementation	Why it is needed
Amazon CloudWatch	Bounded logs, metrics and ten operational alarms	Failure diagnosis and platform health evidence
AWS Budgets and Pricing API	Threshold notifications and captured price inputs	Tracks the agreed service-usage budget and supports the documented estimate
AWS CloudFormation	Bootstrap/platform/simulator templates and stack lifecycle	Reproducible, reviewable infrastructure deployment
AWS CLI v2	All operator AWS operations through a logging wrapper	Executes deployment, verification and recovery while retaining an append-only command audit

No Kinesis, Firehose, QuickSight, IoT Core, external email delivery provider, container runtime or relational database is required. Cognito uses its default AWS email sender. SNS mail and Cognito account mail have separate purposes and confirmation/recovery behavior.

3.2 Application languages and libraries

Version values below come from the committed manifests/lockfile, not a claim about the newest available versions. Frontend values are declared package ranges; the package-lock file records exact resolutions. Python deployed SDK and development lock versions are explicitly pinned.

Technology	Recorded version/constraint	Actual use
Python	Lambda runtime 3.12; local project environment 3.11.8	Backend, simulator, infrastructure generator, deployment and test tooling
boto3 / botocore	Runtime boto3 1.43.113; development lock boto3/botocore 1.43.113	AWS SDK clients, role-based calls, signing and DynamoDB transaction serialization
React / React DOM	`^19.3.1`	Browser component rendering and state
TypeScript	`^7.0.2`	Typed API contracts, component props and build-time checking
Vite / React plugin	`^8.3.4` / `^6.1.2`	Development tooling and production asset bundling
Recharts	`^3.10.1`	Recent telemetry and historical report charts
oidc-client-ts	`^3.5.0`	Cognito authorization code/PKCE, callback and token-session handling
lucide-react	`^1.57.0`	Consistent interface icons
HTML and CSS	Browser standards; project styles/tokens	Responsive layout, entry page, accessibility and reduced-motion behavior
JSON / NDJSON / gzip	Schema version 1; standard Python libraries	API messages, CloudFormation templates and compressed archive files
SQL	Athena workgroup queries	Deduplicated date/motor aggregates with predefined expressions

Node.js is a build/test tool, not the deployed application server. The local authoring machine reports Node v22.14.0. AWS serves the built static files; Lambda runs the Python API. No frontend process needs to stay running on the developer's

computer for the deployment to work.

3.3 Testing, validation and engineering tools

Tool	Recorded manifest/lock version	Use
pytest	Lock 8.4.2	Backend, simulator, CLI audit, infrastructure and live AWS test orchestration
Moto	Lock 5.2.4	Controlled AWS handler integration tests without changing live infrastructure
cfn-lint	Lock 1.57.2	CloudFormation resource/schema validation for the chosen region
CloudFormation Guard	Version 3 tool family	Project infrastructure security/configuration assertions across the three templates
Ruff	Lock 0.17.0	Python linting, import and correctness checks
Playwright Test	`^1.64.1`	Controlled Chromium UI cases and real hosted Cognito/deployed dashboard journeys
axe-core Playwright integration	`^4.13.0`	Automated accessibility checks on tested views
Prettier	`^3.9.10`	Frontend source formatting
Git	Installed local CLI	Incremental source/evidence/documentation commits and reviewable history
PowerShell	Windows shell	Development and CLI/script orchestration
SHA-256 hashing	Python standard library	Event/batch identity and served-release integrity comparisons

Git and the AWS CLI are operational prerequisites; their exact installation versions were not pinned in the project manifests. This report does not invent those values. AWS CLI executable: `C:\Users\aksha\AppData\Local\Programs\Amazon\AWSCLIV2\aws.exe`.

Codex assisted implementation, testing and documentation. The frontend-design skill was used for the revised minimal entry page. These are authoring aids, not deployed AWS services or runtime dependencies. Assignment diagrams are generated vector drawings with matching editable Mermaid text. ReportLab creates the PDF, Pillow supports visual QA, and pdfplumber/pypdfium2 verify text/pages and render review images; these document-generation tools are separate from project application dependencies.

3.4 Source map and reproducibility

Repository location	Responsibility
`PRD.md`	Original functional scope and approved operating constraints
`backend/domain.py`	Validation, canonical hashing, fault counters and predefined SQL
`backend/ingestion.py`	Signed ingestion handler and queue envelope
`backend/processor.py`	Archive, event deduplication, transactional state and alert transitions
`backend/api.py`	Human routes, roles, readings, controls, users and report coordination
`backend/notifications.py`	Subscription awareness and durable pending notification publication
`backend/aws_support.py`	Reused SDK clients, bounded retries, serialization and response helpers
`simulator/main.py`	Smooth synthetic readings, signed batching, remote controls and acknowledgements
`infra/generate.py`, generated JSON and `security.guard`	Infrastructure source, deployable templates and policy assertions
`scripts/aws_cli.py` and `AWS_COMMANDS.txt`	Logged operator CLI execution and immutable append history
`scripts/deploy.py`, `package.py`, `update_simulator.py`	Reproducible release/deployment workflow
`scripts/publish_frontend.py`	Independent static release and CloudFront invalidation
`scripts/recovery.py`	Separate-table bounded reconstruction with no SNS publication
`scripts/budget_model.py`, `cost_estimate.py`	Cost estimate calculations and captured pricing inputs
`tests/` and `tests/live/`	Controlled backend checks and actual AWS acceptance scenarios
`frontend/src/` and `frontend/tests/`	React application and controlled/deployed browser scenarios
`docs/`	API contract, resource outputs, recorded XML/JSON, screenshots and shutdown runbook
`assignment_docs/`	This submission report, source write-ups, diagrams and selected evidence

For project execution, follow the repository README rather than treating the report as a list of commands already run. Typical validation includes `python -m scripts.validate`, `pytest` and the frontend's `npm run build` / `npm test` scripts; live tests must follow their explicit opt-in/environment requirements. New AWS CLI calls must use the logger, verify the authorized account/region and respect continued operation. Rebuilding these submission documents does not perform AWS operations.

3.5 References

Implementation facts are grounded in the repository files listed above, the current PRD and recorded evidence. AWS documentation links below explain the underlying service behavior; they are not evidence that an unrecorded deployment/test happened.

- AWS Lambda with SQS: https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html
- SQS partial batch responses: https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-errorhandling.html
- DynamoDB transactions: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transactions.html
- DynamoDB TTL behavior: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html
- CloudFront private S3 origin access: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html
- Cognito authorization endpoint/PKCE: https://docs.aws.amazon.com/cognito/latest/developerguide/authorization-endpoint.html
- Cognito email settings: https://docs.aws.amazon.com/cognito/latest/developerguide/user-pool-email.html
- Cognito password-reset error behavior: https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pool-managing-errors.html
- Athena workgroup scan limits: https://docs.aws.amazon.com/athena/latest/ug/workgroups-setting-control-limits-cloudwatch.html
- Athena partition projection: https://docs.aws.amazon.com/athena/latest/ug/partition-projection.html

The pinned manifests and local source remain the authority for this implementation. Future service/library changes may require revalidation.

Part 4. Verification evidence and screenshots

4.1 Evidence provenance

The submission packages existing implementation records, rather than running another disruptive acceptance exercise. AWS-stage evidence is dated 14 October 2026; the entry-page refinement, final UI evidence and account diagnostics extend into 15 October in India. Machine timestamps in JSON/XML may use UTC. Relative links below work from the repository; copies of the compact summaries are included under `evidence/` for convenient submission.

Evidence	What to inspect	Location
AWS acceptance summary	Fourteen distinct scenario names, status, source run and measurements	Copied JSON, original XML
UI acceptance summary	Twenty-three controlled cases, four earlier deployed journeys, later smoke and release hashes	Copied JSON
Full deployed browser run	Four journey count and real run outcome, including setup/cleanup	Original XML
Final published smoke	Real desktop/mobile hosted-login and live chart result	Copied JSON, original XML
Deployed health/config snapshot	Three motor states, mapping/queues, alarms, private storage, retention and budgets at capture	Copied JSON
Fault notifications	Stored SNS acceptance records; human receipt is separately reported in acceptance notes	Copied JSON
Historical cost model	Inputs, estimated component amounts and explicit assumptions	Copied JSON, pricing snapshot
Deployment outputs	CloudFormation outputs and deployed logical-to-physical resources	Original JSON
Account access exception	Latest unresolved first-password/recovery diagnostic and investigation boundary	Current handoff
Operator command audit	Timestamp, purpose, non-secret command arguments and exit status	AWS_COMMANDS.txt
Future cleanup	Ordered shutdown and retained-resource handling; live teardown not run	SHUTDOWN.md

The copied `ui-acceptance.json` contains an earlier invitation-recovery note. The later unresolved password report and last diagnostic in `PROGRESS.md` supersede any implication that the user's sign-in is complete. The document-build manifest hashes the source documents, generated artifacts and selected evidence; it establishes file integrity, not the correctness of an AWS service outside the recorded run.

4.2 Scenario coverage

Backend checks include strict input validation, rule boundaries/counters, retry-safe archive/transactions, duplicate and old events, controls, authorization, report bounds/SQL, command-audit restrictions and infrastructure invariants. Moto-backed integration tests exercise handler interactions in a controlled environment; they are distinct from real AWS scenarios.

The fourteen live AWS scenarios cover healthy EC2 operation and fault recovery; signed invalid ingestion; deduplication, ordering and archive; consumer interruption/catch-up; poison-message DLQ and corrected redrive; real tokens, roles and hosted PKCE; user management/revocation; bounded burst and stopped reporting; confirmed notification acceptance/retry suppression; bounded Athena/private hosting; known-data report calculations; cache hit/update invalidation; and limited object/table recovery.

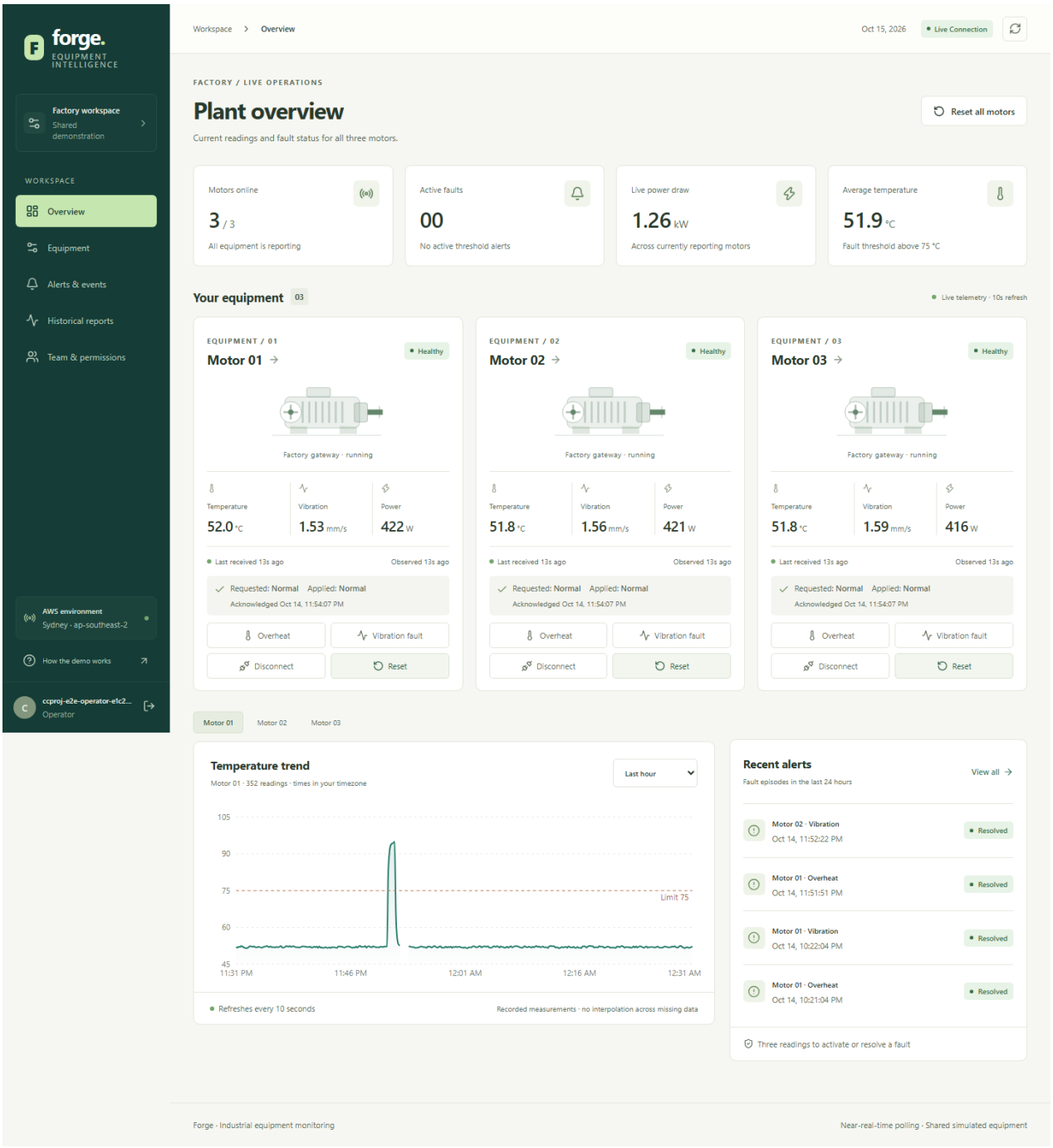
Controlled Playwright cases cover failure and rare states reproducibly, including expired sessions, failed renewal, bad OAuth state, retained stale responses, timed-out controls, offline/stopped differences, report empty/error outcomes, role/self-protection, accessible controls and mobile overflow. The real browser journeys use the deployed Cognito/API and actual measurements, with disposable test accounts removed afterward. Invitations are suppressed for disposable live accounts; controlled invitation UI tests avoid sending unsolicited mail.

Recorded AWS-stage latency was 5.063 seconds from signed ingestion to current state, including polling. The limited recovery measured 70.641 seconds before verification/cleanup. No percentile performance claim, production load ceiling, full-regional recovery objective or universal functionality guarantee is inferred from those samples.

4.3 Selected visual evidence

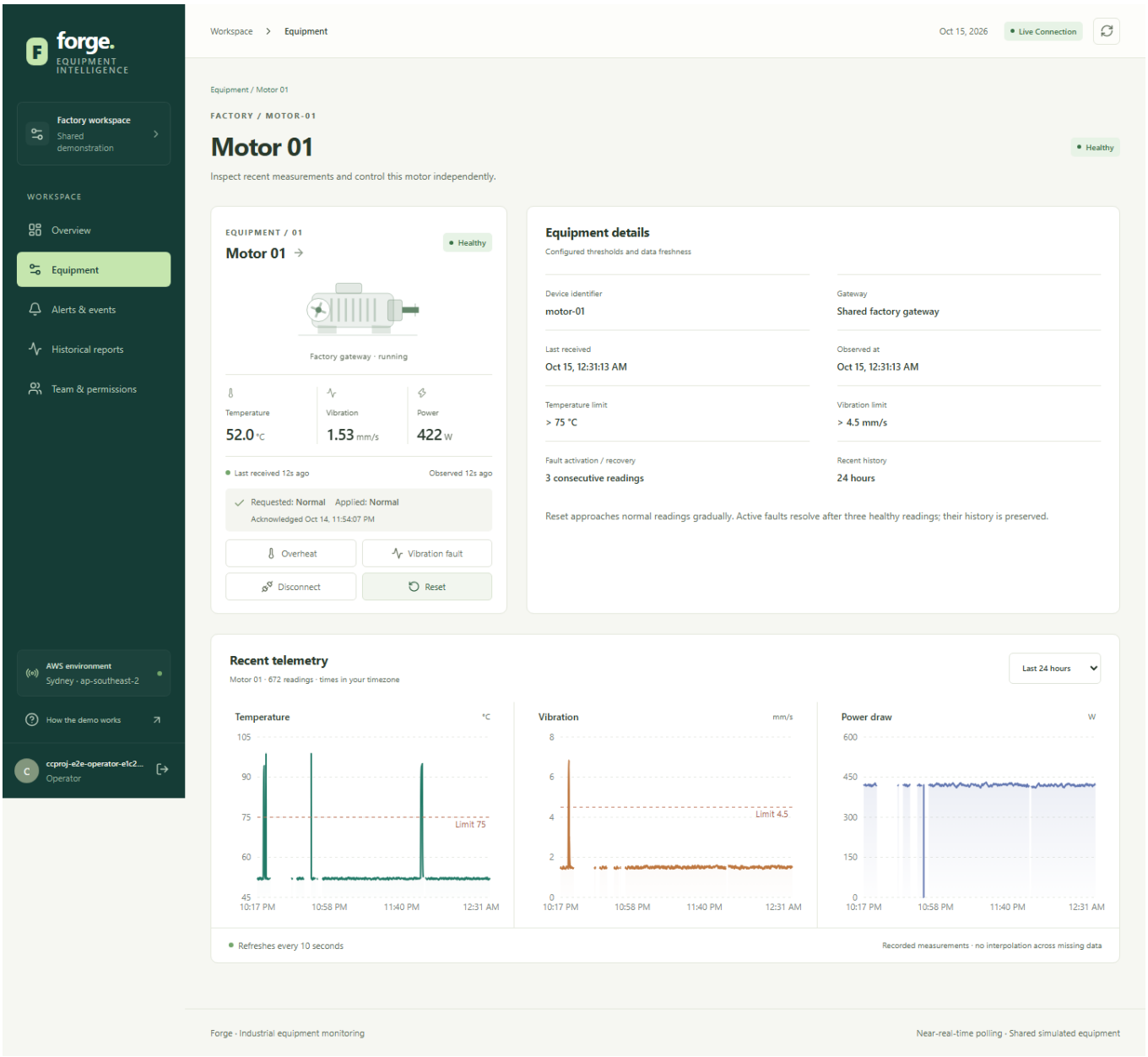
The overview, detail, alerts, reports and mobile overview below were captured from the deployed application during real browser testing. Visible users are disposable test accounts and history contains deliberate test faults/readings. They demonstrate the interface and tested data, not uninterrupted normal industrial operation. Login images show the later revised entry layout captured by controlled browser tests; they do not show credentials or a successful initial-account password setup.

Screenshot A. Desktop motor overview



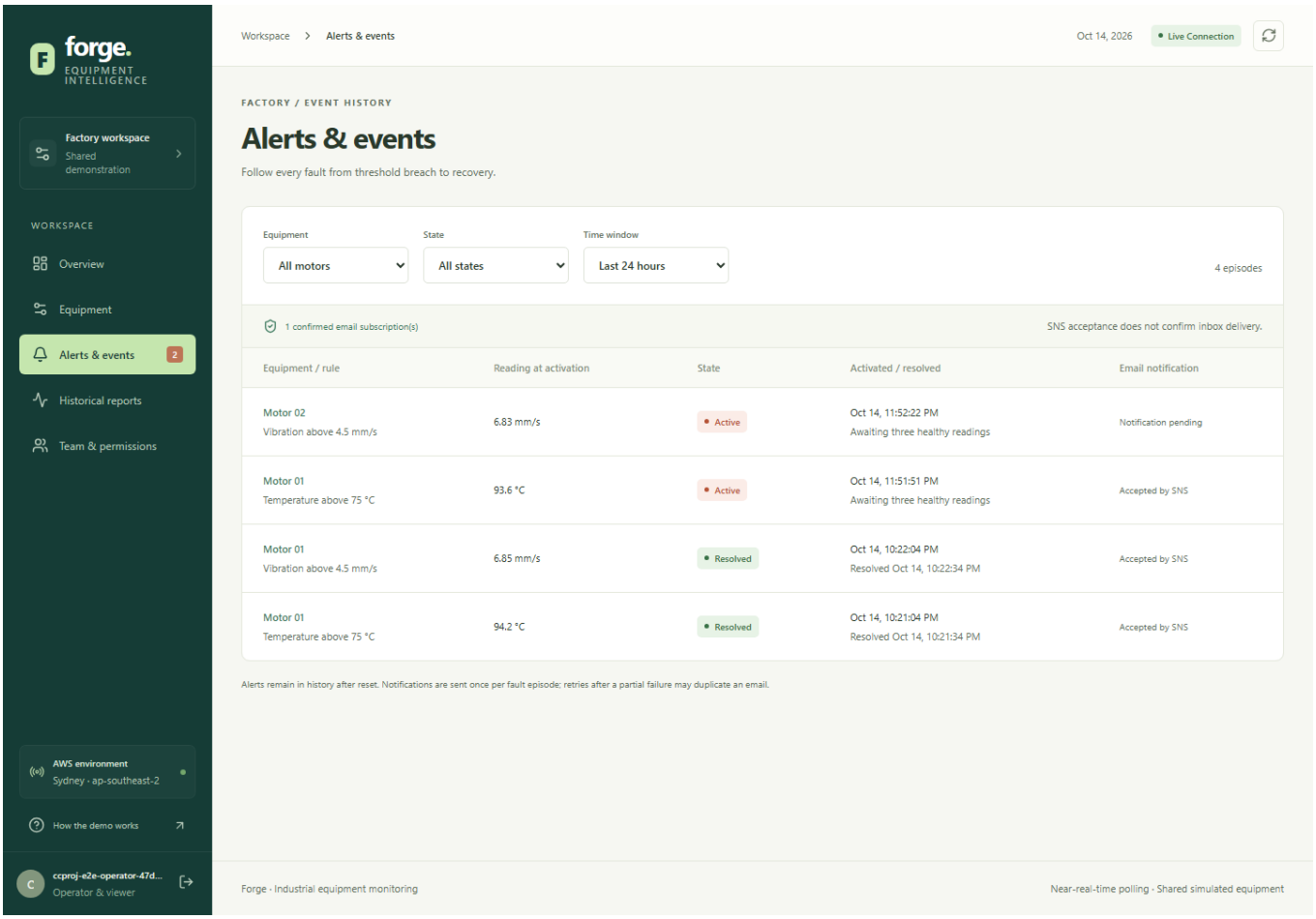
Desktop overview captured during deployed browser acceptance

Screenshot B. Recent temperature, vibration and power



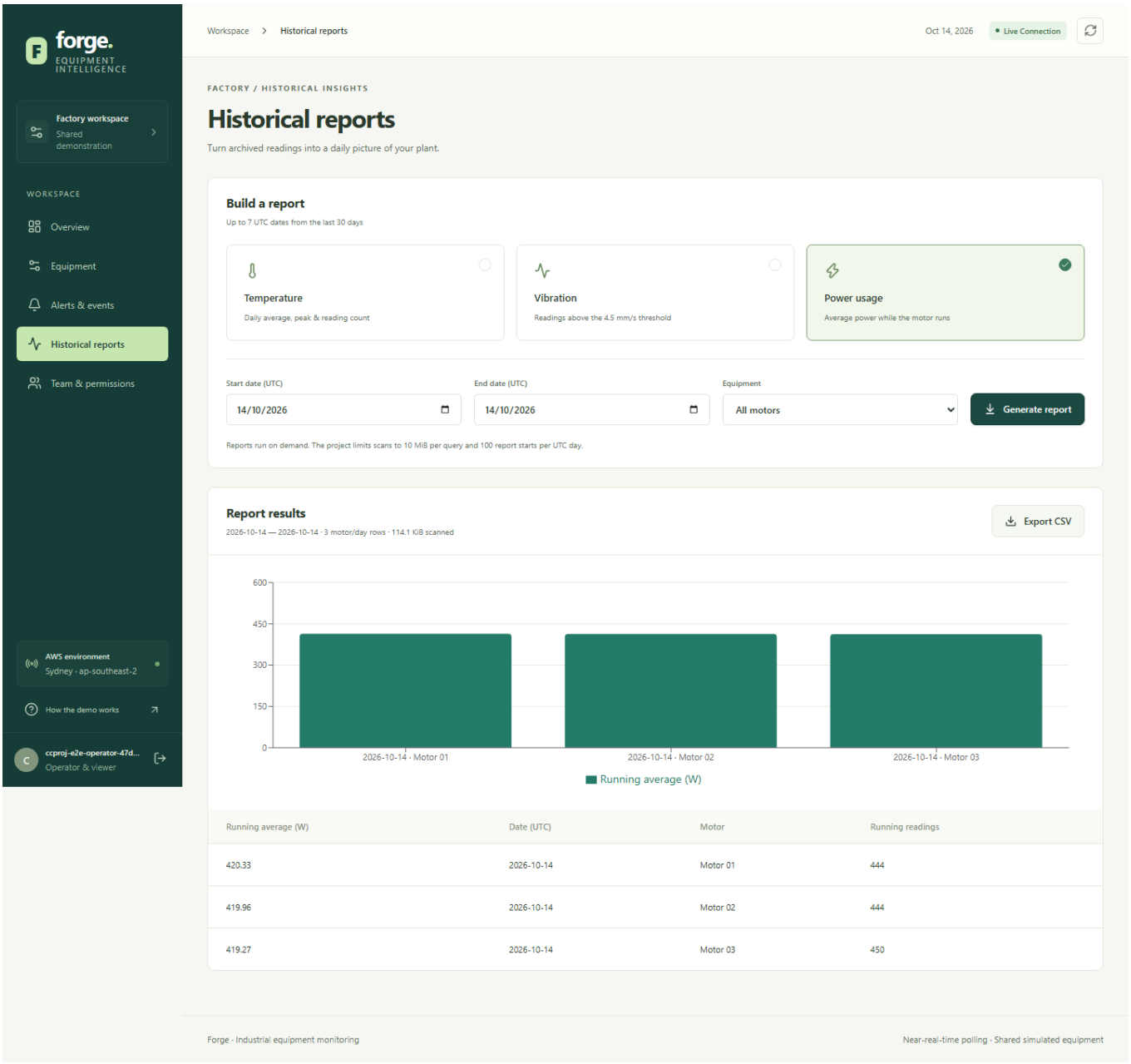
Device detail charts captured during deployed browser acceptance

Screenshot C. Active and resolved fault history



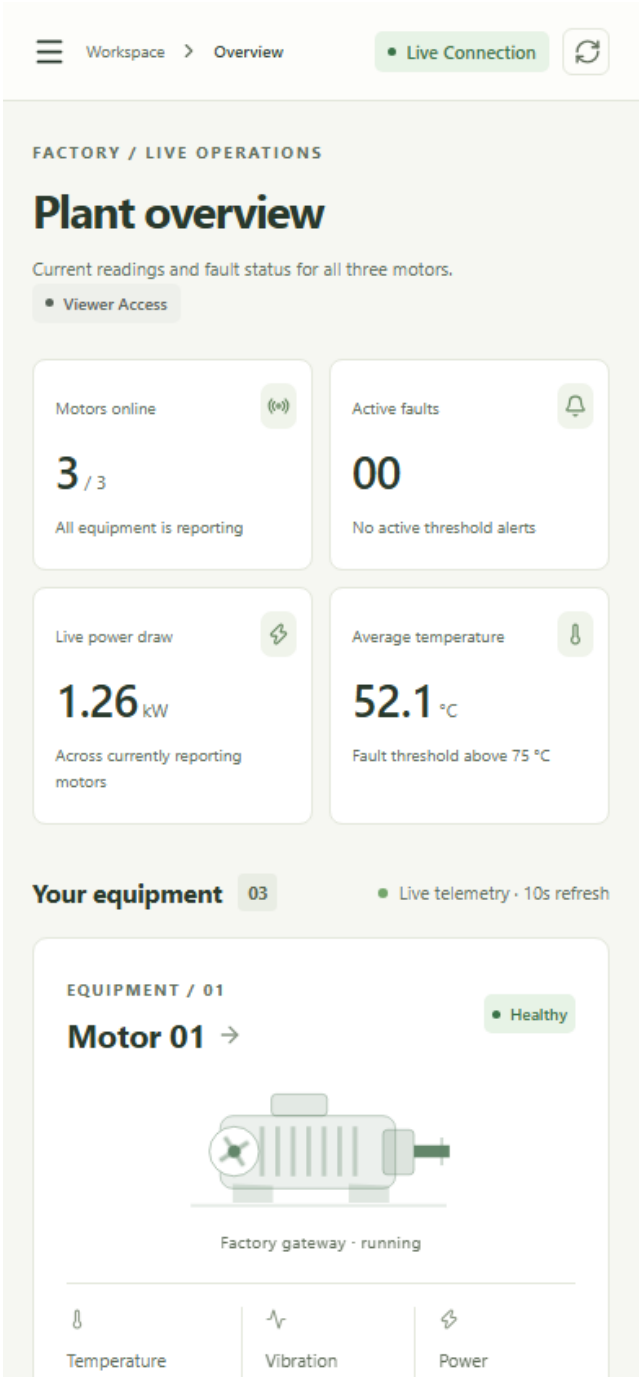
Alert view captured during deployed browser acceptance

Screenshot D. Historical report results



Report view captured during deployed browser acceptance

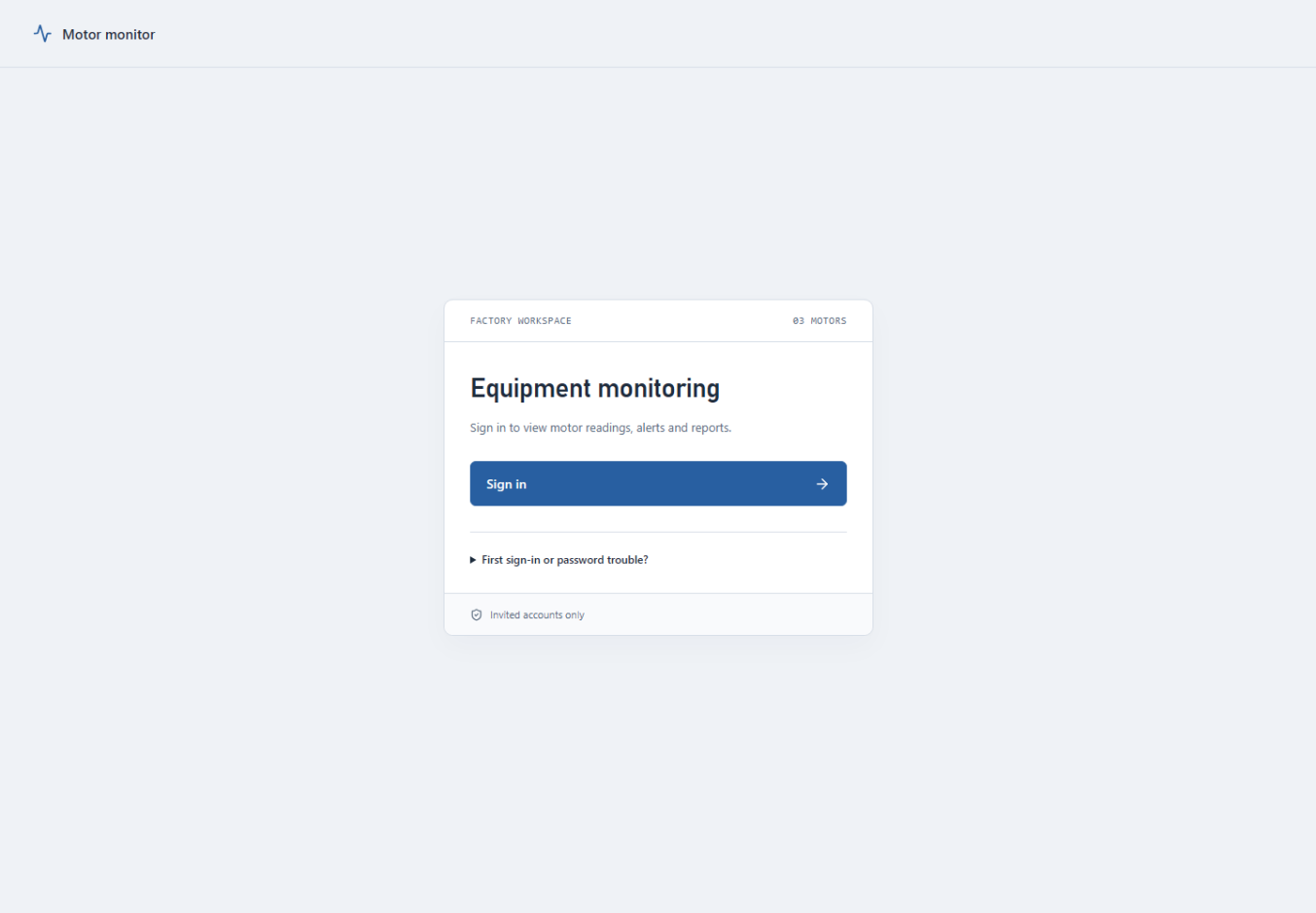
Screenshot E. Mobile viewer layout



Mobile overview viewport excerpt from deployed browser acceptance

The top viewport is shown for print legibility. The unchanged full-page mobile capture is also supplied.

Screenshot F. Revised concise entry page



Latest sign-in entry layout captured with controlled browser testing

4.4 Demonstration sequence for an evaluator

1. Resolve the initial-account password issue, then sign in with an authorized evaluator account. Explain the distinct viewer/operator roles.
2. Show three normally reporting motors, freshness timestamps and recent charts. Explain that the gateway produces synthetic readings.
3. Request overheat on one motor. Show pending then applied acknowledgement, gradual measurement change and an active alert after three breached readings. Observe that the other motors continue normally.
4. Show retained alert history and the recorded confirmed-email evidence. Do not equate a new `sns_accepted` label with verified inbox delivery.
5. Reset the motor and observe recovery/three healthy readings. Demonstrate disconnect versus a stopped-but-reporting reading using the existing test evidence if another live fault is unnecessary.
6. Run one bounded historical report and explain date projection, deduplication and scan bounds. Use the saved known-data test result to support calculation correctness.
7. Show the queue-interruption/DLQ/recovery evidence and CloudFormation source. A new live failure injection is optional and must follow the safe test procedures.
8. Present the estimate versus actual-spend distinction, current acceptance exception, command audit and future shutdown procedure. Leave the project online under the existing instruction.

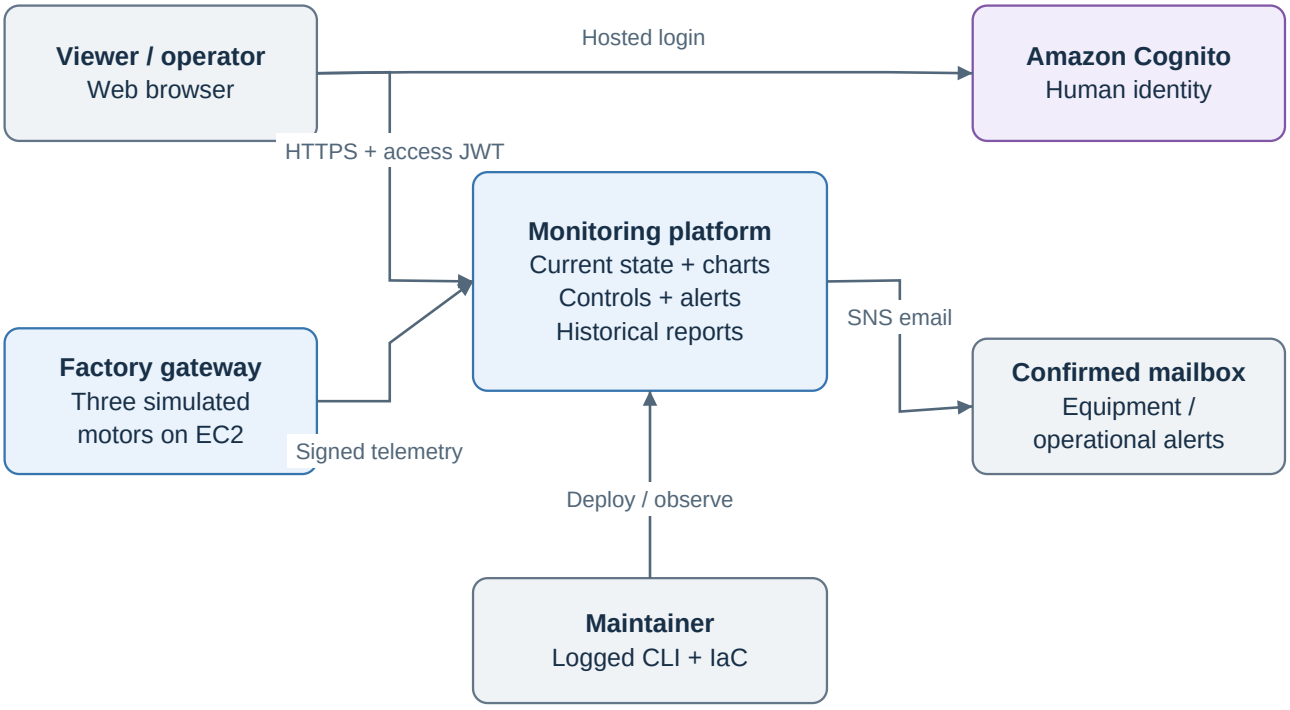
4.5 Submission readiness record

Requirements, implemented design, tooling inventory, architecture diagrams, selected UI evidence and reproducibility references are provided. Source and evidence have been committed incrementally. This report does not mark the initial account's first access as resolved and does not invent a final actual cost. Those are explicit follow-up acceptance items.

Part 5. Architecture and design diagrams

Ten diagrams describe the implemented system at complementary levels. Arrows show request, data or dependency direction as labeled. Boxes are logical responsibilities unless a network boundary is explicitly drawn. Full-size SVG/PNG and editable Mermaid versions accompany the report.

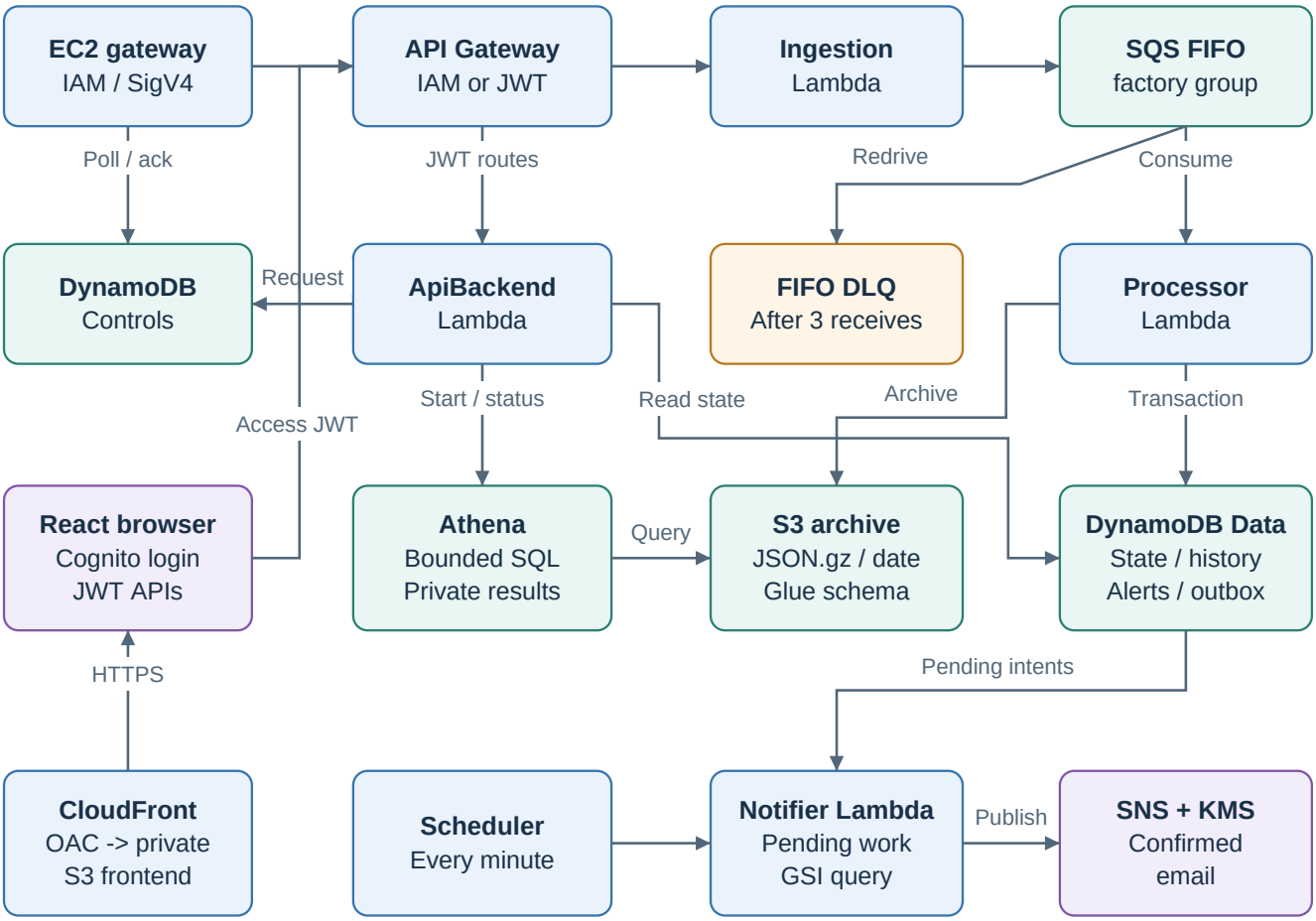
Figure 1. System context



Regional deployment: Sydney. CloudFront delivers the static browser application at the edge.

People authenticate through Cognito; the gateway uses IAM. Equipment notification subscription is separate from a login account.

Figure 2. Implemented AWS components

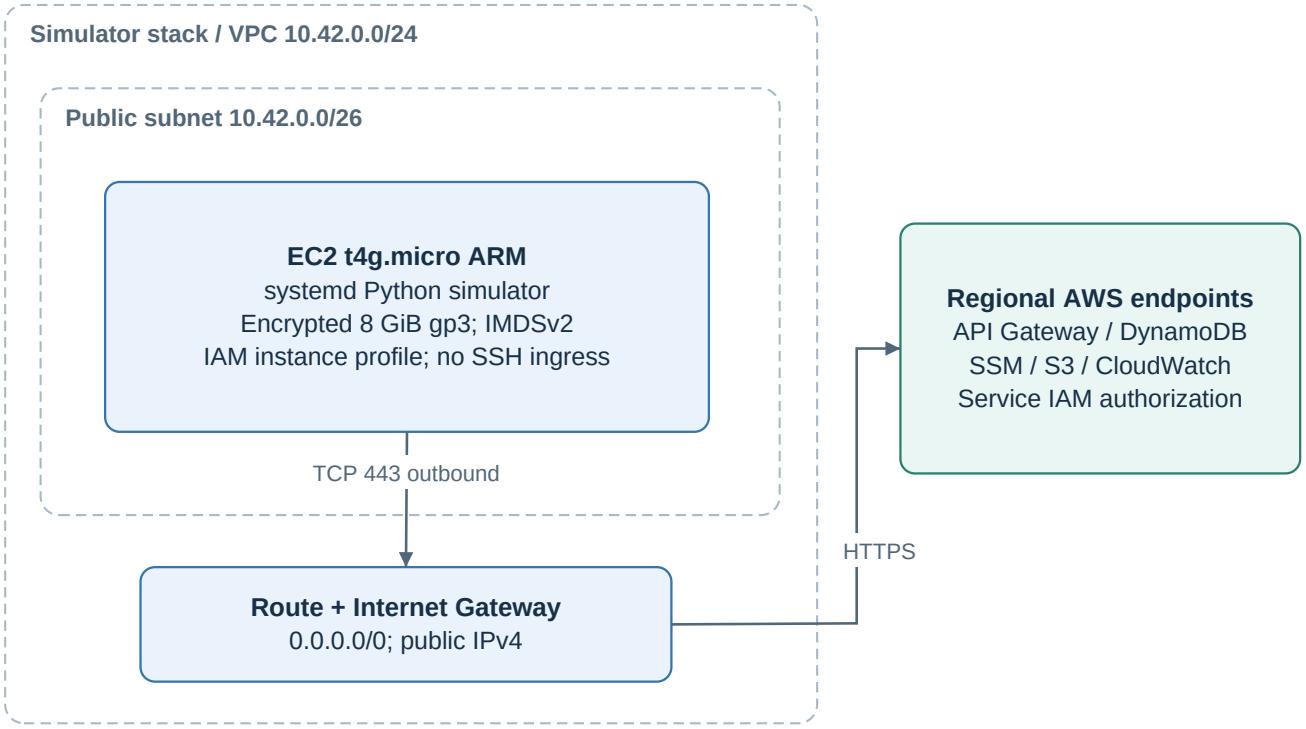


Cross-cutting: IAM roles, CloudWatch logs/alarms, CloudFormation and logged AWS CLI.

Four Lambda functions. Live reads use DynamoDB; only requested reports use Athena.

Logical data flow, not VPC placement. ApiBackend owns controls and reporting. A durable outbox and scheduled Notifier separate email from telemetry commit.

Figure 3. Network and runtime boundary

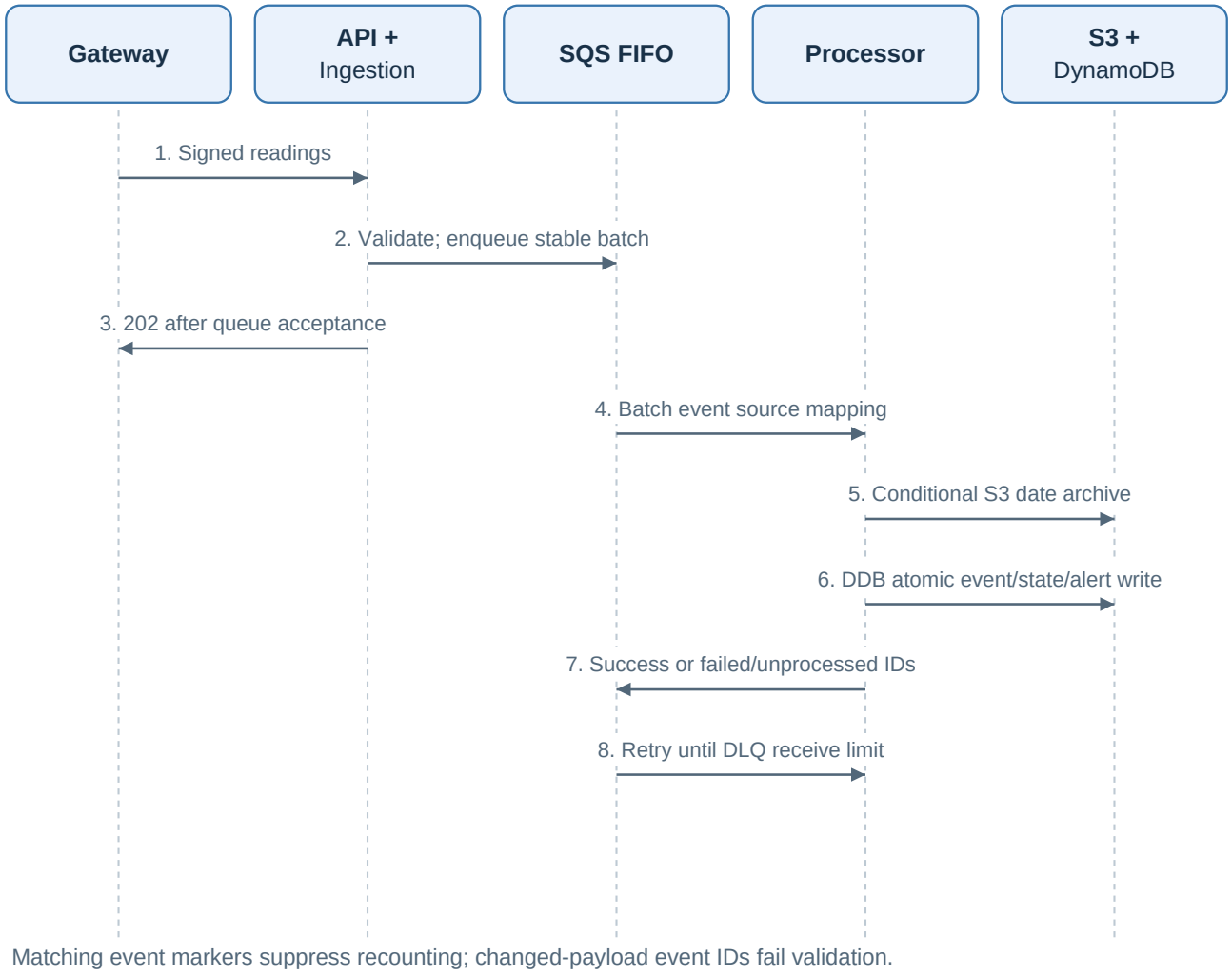


DNS egress: TCP/UDP 53 to the VPC resolver 10.42.0.2.

No NAT Gateway, inbound listener, bastion host or continuously running frontend server.

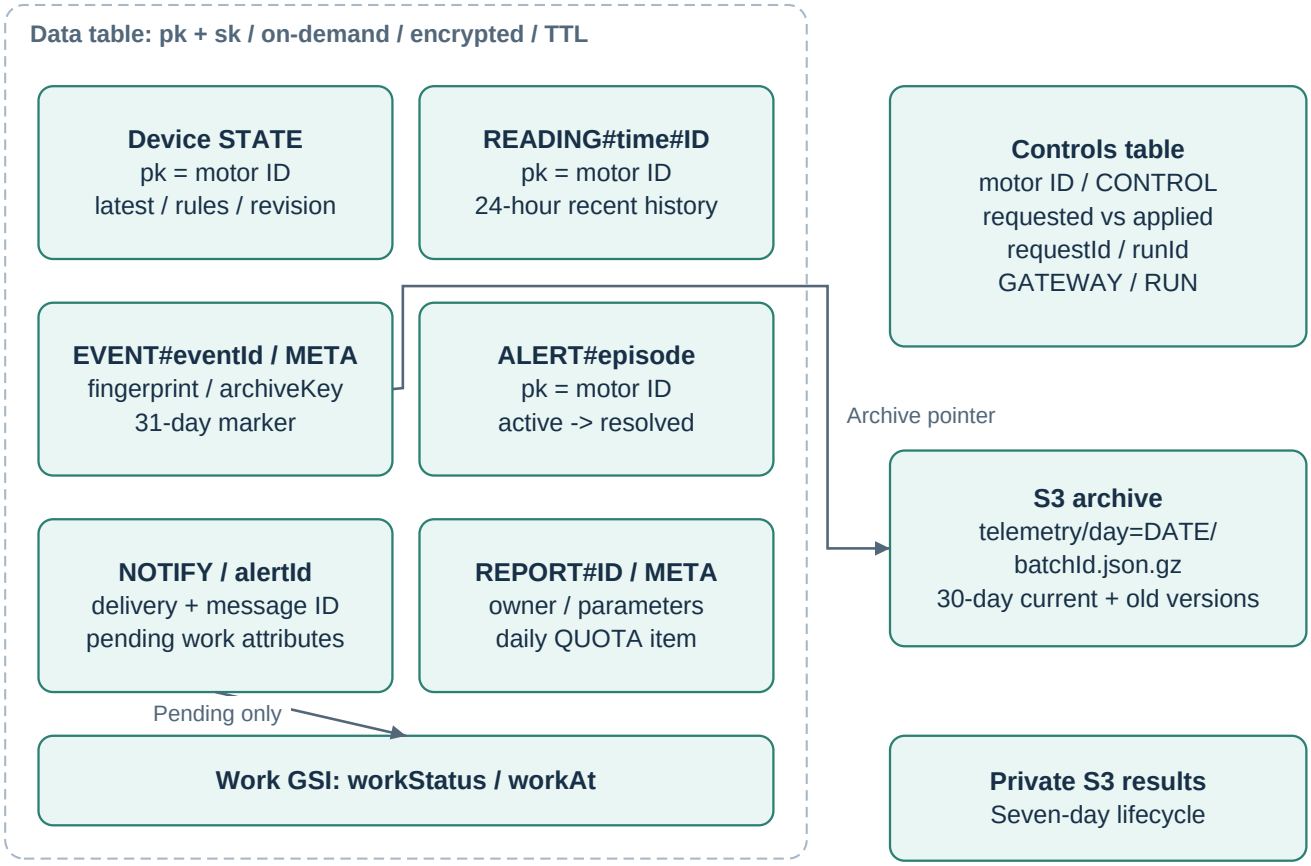
Only the simulator occupies the project VPC. Public IPv4 supports outbound HTTPS; its security group has no inbound rules. Lambda is not placed in this VPC.

Figure 4. Telemetry commit and retry sequence



A 202 confirms queue acceptance. Processor archives before per-reading transactions. If a message fails, FIFO processing stops and unprocessed IDs are returned for retry.

Figure 5. Logical data model and access patterns

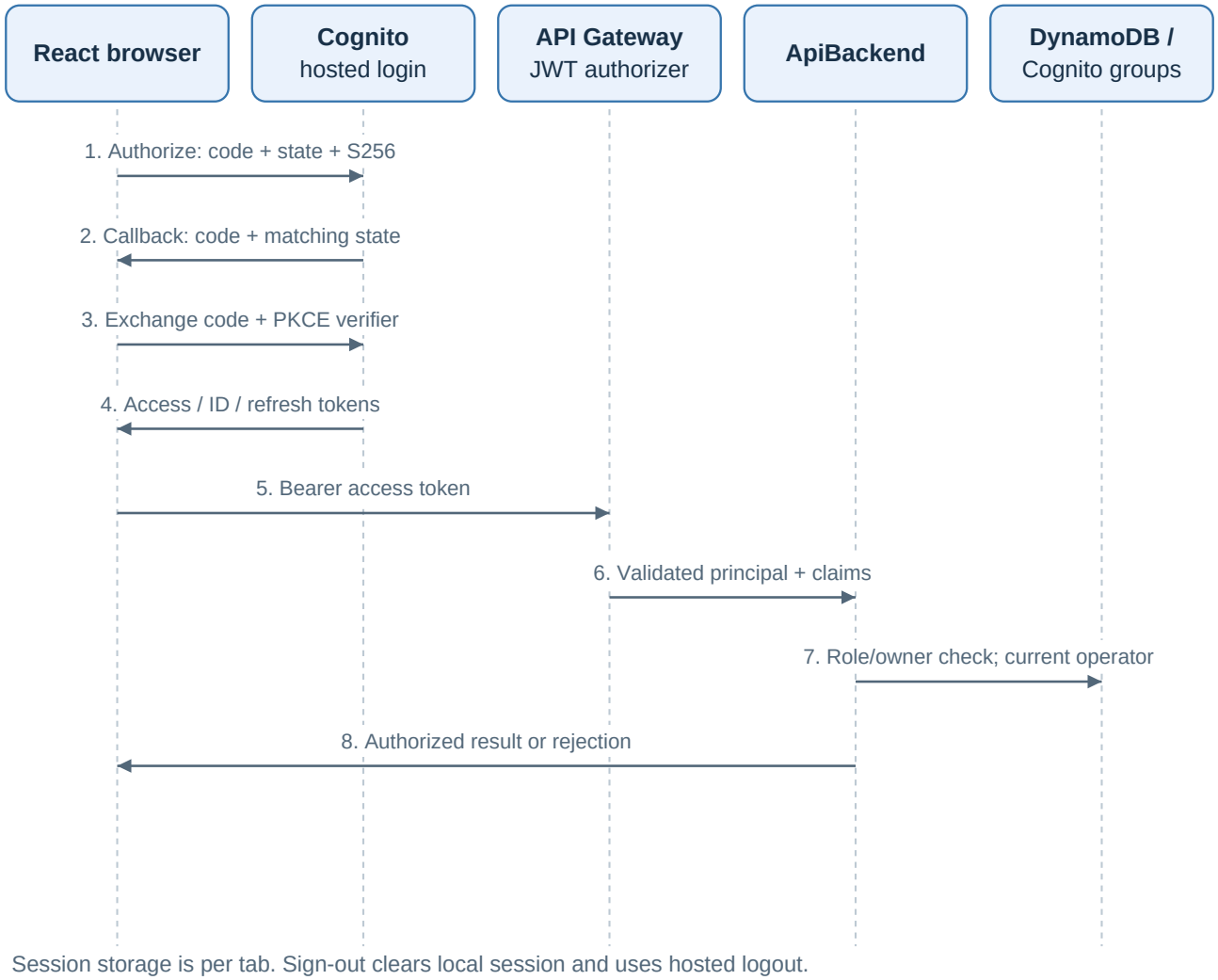


API date bounds enforce visibility even while asynchronous TTL deletion is pending.

Device state and control records remain current; history items have separate lifetimes.

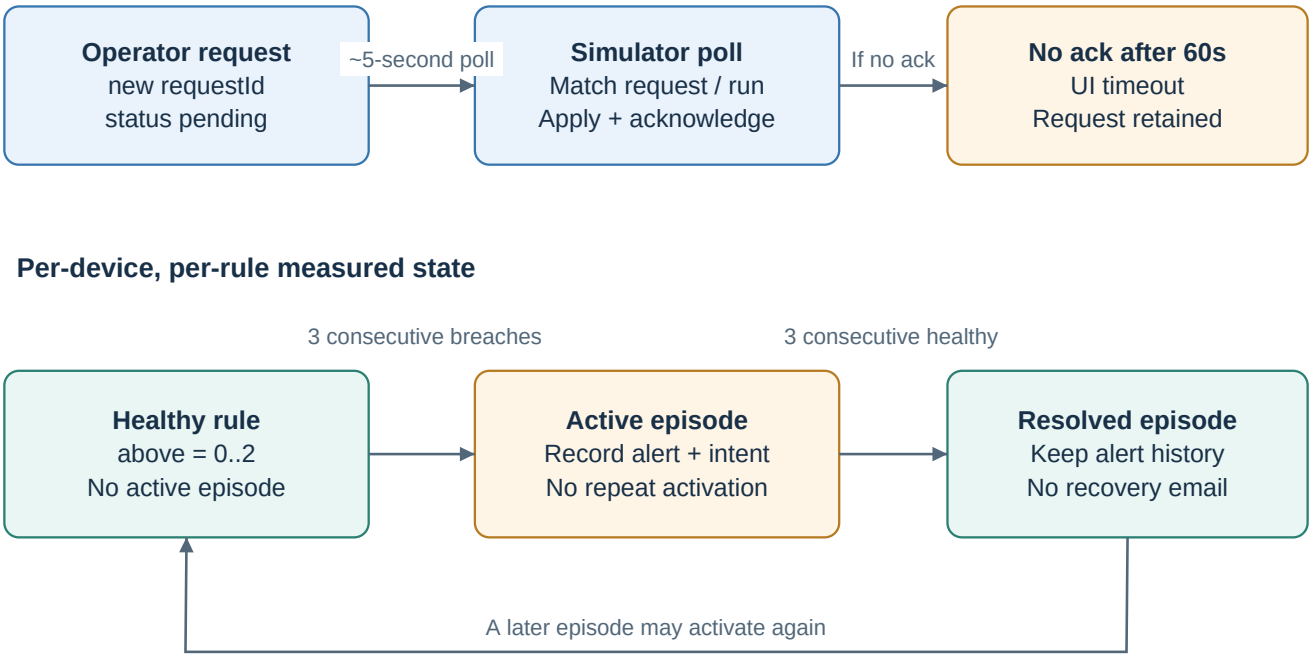
These are DynamoDB item types and S3 layouts, not relational tables. pk/sk compose each item key; a sparse index selects pending notification work.

Figure 6. Browser authentication and authorization



The browser is a public client with no client secret. The access token authorizes application APIs; privileged backend requests also check current Cognito groups.

Figure 7. Controls and fault state machines



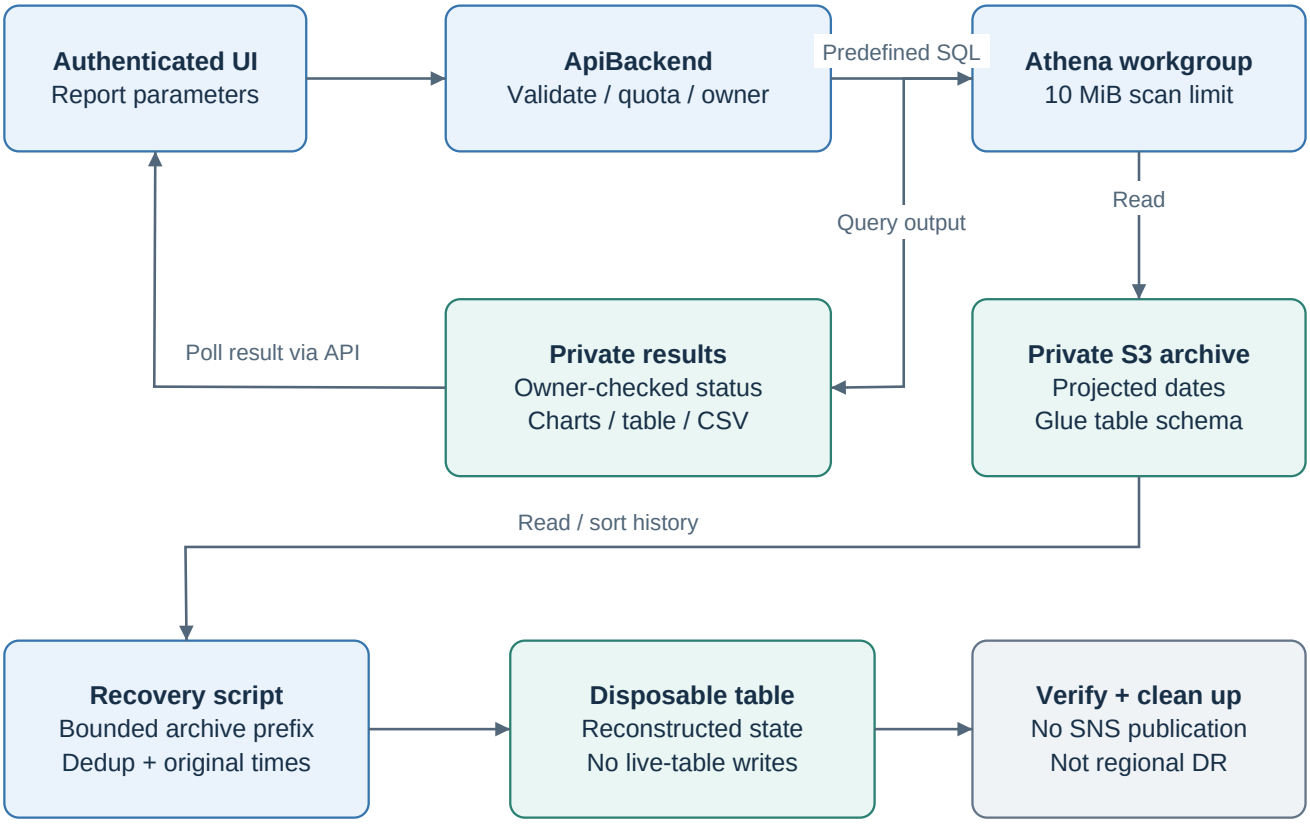
Temperature >75 C; vibration >4.5 mm/s. Each opposite reading resets its sequence.

Stopped + reporting can be online. No valid processed receipt for 60s means offline.

Only newer observations advance counters. Operator reset does not delete alerts.

Control acknowledgement and measured fault state are different processes. Reset changes the requested simulator mode; actual healthy readings resolve an alert.

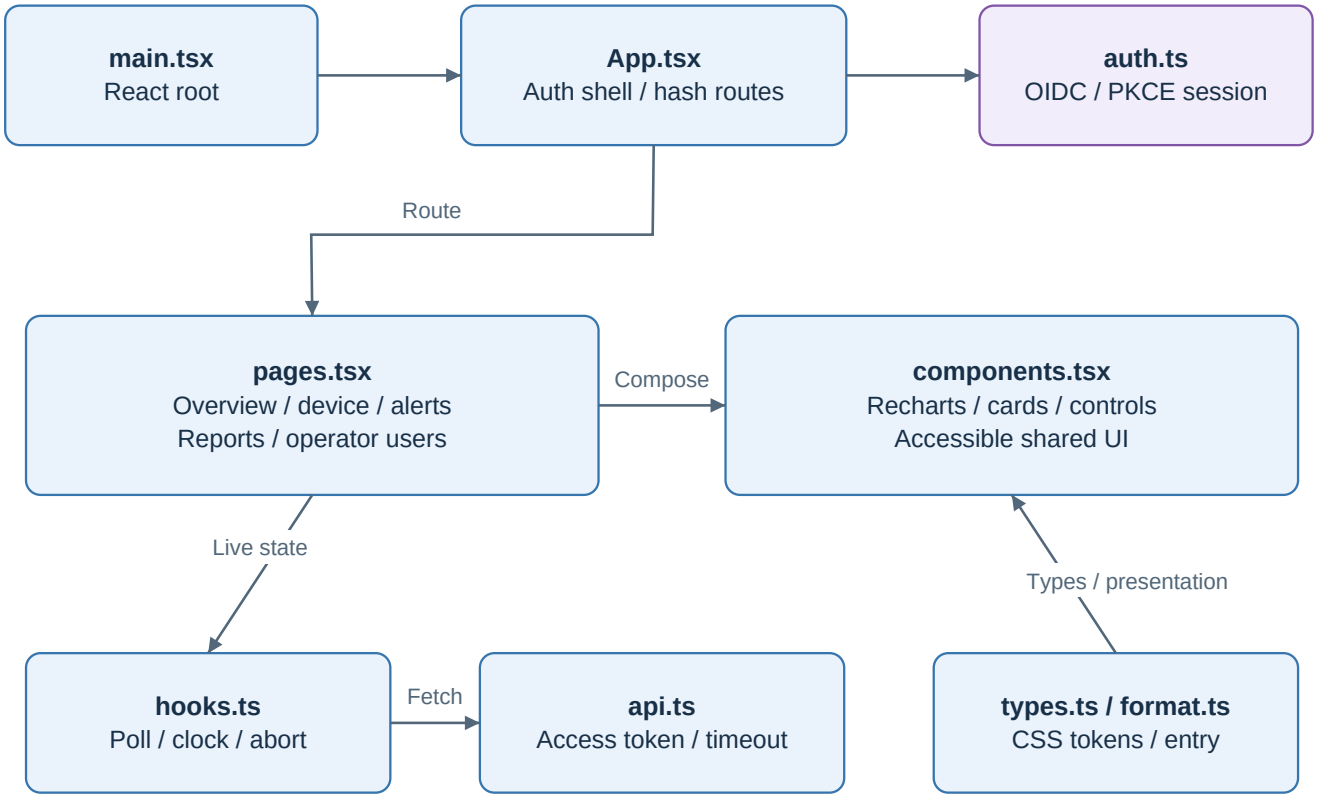
Figure 8. Historical reports and limited recovery



Reports: 1-7 dates within 30 days, at most 100 project report starts per UTC day.

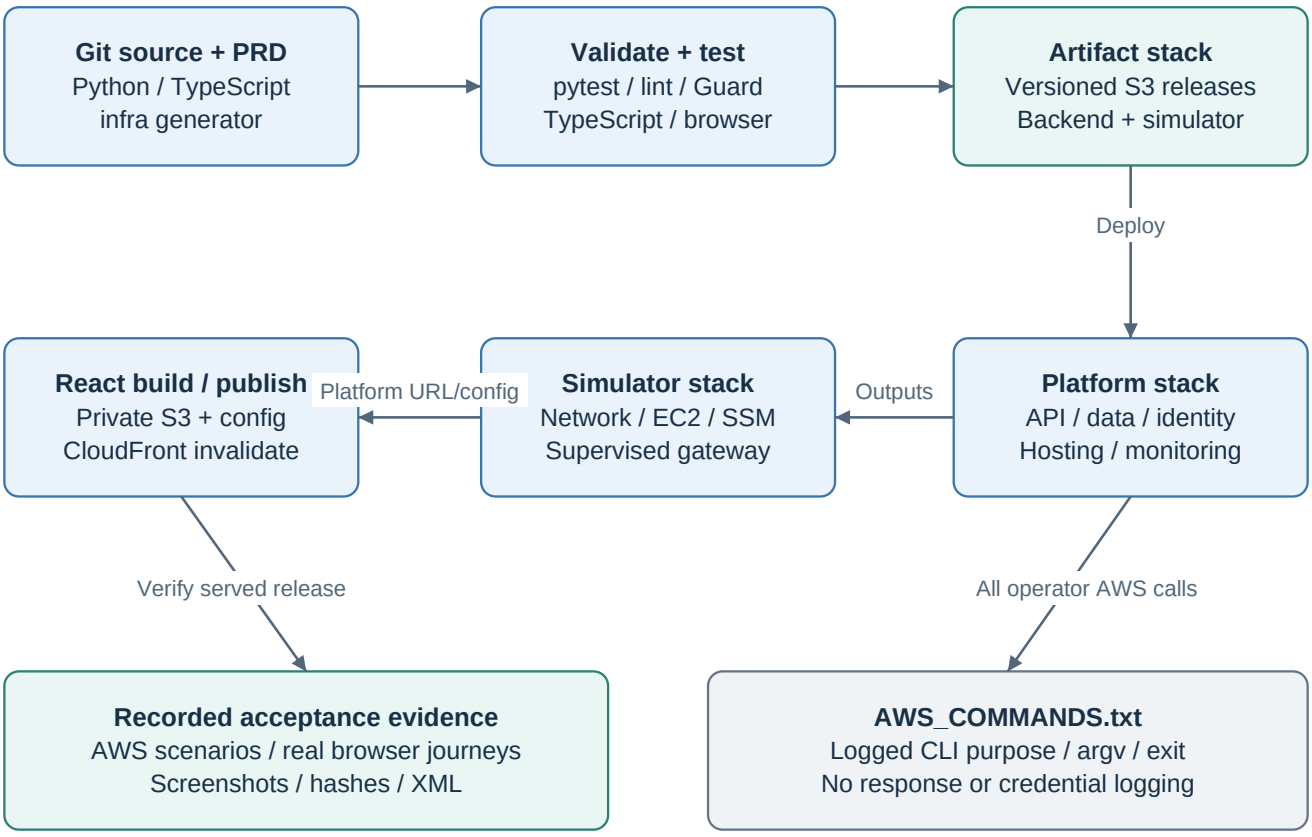
Reporting and reconstruction reuse private history for different purposes. Reconstruction targets a disposable table and never republishes old fault emails.

Figure 9. Frontend software structure



The browser serves five role-aware views. Shared polling and authentication feed a single protected API client; charts and UI state are derived from API responses.

Figure 10. Build, deployment and evidence lifecycle



Infrastructure and static UI have independent release steps. Operator AWS calls use the append-only logger; the document build is an offline authoring step, not another deployment.